

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

**«Белорусский государственный университет информатики и
радиоэлектроники»**

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ

по лабораторной работе №1

по дисциплине

**«Современные технологии программирования масштабируемых
приложений»**

**«Микросервис ввода и модификации данных о клиентах банка. Модуль
«Клиенты»»**

Выполнил:

Лебедев Артем Владимирович
магистрант группы 556301

Преподаватель:

Хмелёва Ангелина Владимировна
кандидат технических наук
доцент кафедры ПОИТ

Минск, 2026 г.

СОДЕРЖАНИЕ

1	Цель работы	1
2	Задание	2
3	Проектирование	3
3.1	Архитектура	4
3.2	Схема данных	4
3.3	REST-интерфейс	5
4	Реализация	6
4.1	Структура проекта	7
4.2	Web-клиент	7
4.3	Двойная проверка маски и обязательности	7
4.4	Уникальность паспорта и идентификационного номера	10
5	Тестирование	10
5.1	Уровни тестов	11
5.2	Программа защиты в виде автоматических тестов	11
5.3	Результаты прогона	13
6	Результаты работы	13
	Заключение	19
	Список использованных источников	20
	Приложение А Исходный код микросервиса «Клиенты»	21

1 ЦЕЛЬ РАБОТЫ

Разработать учебную подсистему ввода и модификации данных о клиентах условного банка: спроектировать схему данных со справочниками, реализовать микросервис с REST-интерфейсом на Java и Spring, web-клиент с формами «Список клиентов» и добавления, редактирования и удаления клиента, обеспечить проверку вводимых значений дважды — в браузере и в сервисе — и покрыть разработанное ПО модульными, интеграционными и функциональными тестами.

2 ЗАДАНИЕ

Работа выполнена по варианту № 4. Номер варианта определяет состав полей клиента: вариант чётный, поэтому в форму входит поле «Пол» и не входят «Место работы» и «Должность»; $4 \bmod 3 = 1$, поэтому из трёх полей «Город прописки», «Адрес прописки» и «Военнообязанный» реализуется первое. Итоговый состав полей приведён в таблице 2.1.

Таблица 2.1 – Поля клиента для варианта № 4

Поле	Тип	Обяз.	Контроль значения
Фамилия, имя, отчество	текстовый	да	только буквы
Дата рождения	дата	да	существующая дата, не позднее сегодня
Пол	radio	да	мужской или женский
Серия паспорта	текстовый	да	две латинские буквы
№ паспорта	текстовый с маской	да	7 цифр, уникален вместе с серией
Кем выдан	текстовый	да	не пусто
Дата выдачи	дата	да	не раньше даты рождения
Идентификационный номер	текстовый с маской	да	3140301A001PB5, уникален
Место рождения	текстовый	да	не пусто
Город проживания	список	да	значение справочника
Адрес проживания	текстовый	да	не пусто
Телефон домашний	текстовый с маской	нет	293-88-44
Телефон мобильный	текстовый с маской	нет	+375 (29) 314-15-92
E-mail	текстовый	нет	формат адреса
Город прописки	список	да	значение справочника
Семейное положение	список	да	значение справочника
Гражданство	список	да	значение справочника
Инвалидность	список	да	значение справочника
Пенсионер	checkbox	да	да или нет
Ежемесячный доход	денежный	нет	неотрицательная сумма

Требования задания:

1. разработать схему данных со справочниками для элементов типа «Список» и реализовать её в СУБД, работающей в памяти; база заполняется автоматически при старте;
2. создать микросервис для REST-подключения к базе данных;
3. разработать web-клиент с формой «Список клиентов», отсортированной по фамилиям, и формой добавления, удаления и редактирования клиентов;
4. проверять значения каждого поля дважды — на клиенте и в REST;
5. добавить в базу не менее пяти клиентов, включая себя;
6. протестировать ПО на WebDriver с базой H2 в памяти по пунктам программы защиты.

3 ПРОЕКТИРОВАНИЕ

3.1 Архитектура

Модуль реализован как самостоятельный микросервис: у сервиса `client-service` своя база данных, собственный REST-интерфейс и собственный web-клиент, который раздаётся тем же процессом как статический ресурс. Остальные модули банка (депозиты, кредиты) получают сведения о клиентах только через REST, поэтому сервис можно развернуть, остановить и масштабировать независимо от них. Структура сервиса показана на рисунке 3.1.

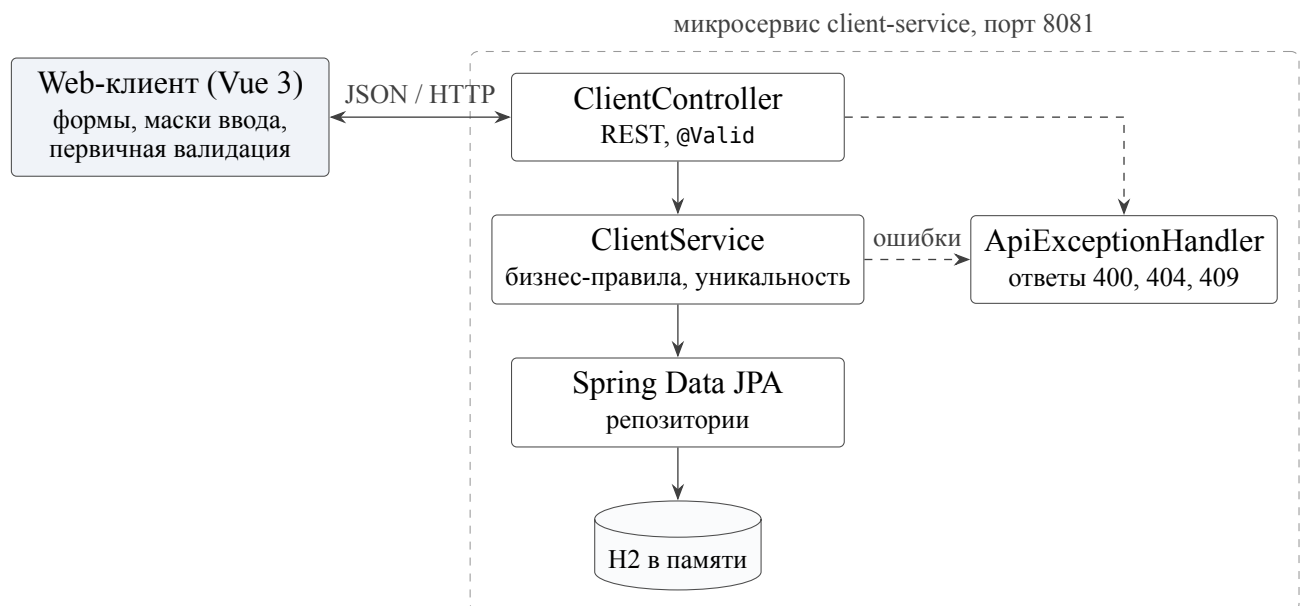


Рисунок 3.1 – Структура микросервиса «Клиенты»

Использованные технологии: Java 17, Spring Boot 3.5 (Spring MVC, Spring Data JPA, Bean Validation), H2 в режиме in-memory, Vue 3 на стороне браузера, JUnit 5, Mockito, MockMvc и Selenium WebDriver в тестах, Maven в качестве системы сборки. Библиотека Vue подключается из webjar и раздаётся самим сервисом, поэтому для запуска не нужны ни Node.js, ни доступ в интернет.

3.2 Схема данных

Каждое поле типа «Список» вынесено в отдельный справочник: `city` (используется дважды — для города проживания и города прописки), `marital_status`, `citizenship` и `disability`. Таблица `client` ссылается на справочники внешними ключами (рисунок 3.2). Пол хранится символом M/F с ограничением `check`, поскольку набор значений фиксирован и справочник для него не нужен.

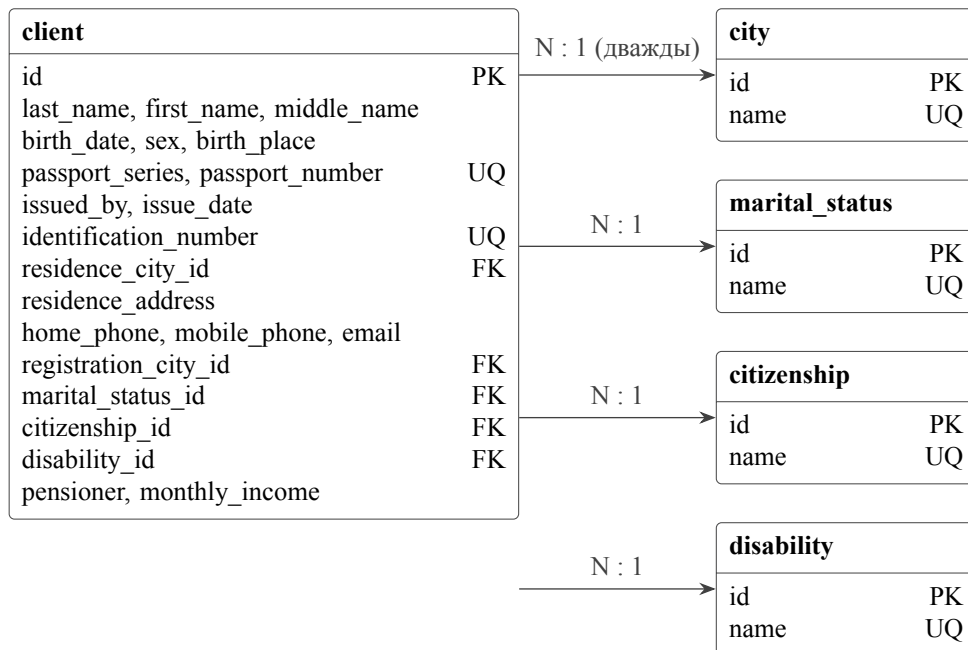


Рисунок 3.2 – Схема данных модуля «Клиенты»

Схема создаётся сценарием `schema.sql`, справочники и шесть тестовых клиентов (включая автора работы) загружаются сценарием `data.sql` при каждом старте приложения. Hibernate работает в режиме `ddl-auto=validate`: он не создаёт таблицы сам, а сверяет сущности с явно описанной схемой и не даст сервису стартовать при расхождении. Ограничения уникальности вынесены в саму базу (листинг 3.1) — это последний рубеж защиты от дублей, который срабатывает даже в обход сервиса.

Листинг 3.1 – Фрагмент `schema.sql`: ссылки на справочники и уникальность

```

create table client (
    id                bigint generated by default as identity primary key,
    last_name         varchar(60) not null,
    ...
    sex              varchar(1) not null check (sex in ('M', 'F')),
    residence_city_id bigint not null references city,
    registration_city_id bigint not null references city,
    marital_status_id bigint not null references marital_status,
    citizenship_id    bigint not null references citizenship,
    disability_id     bigint not null references disability,
    pensioner         boolean not null,
    monthly_income    decimal(14, 2),
    constraint uq_client_passport unique (passport_series, passport_number),
    constraint uq_client_identification unique (identification_number)
);

```

3.3 REST-интерфейс

Сервис предоставляет шесть операций (таблица 3.1). Данные передаются в JSON, даты — текстом в формате ДД.ММ.ГГГГ: это позволяет серверу самому решать, является ли присланная строка датой, и возвращать ошибку

конкретного поля, а не общий сбой разбора запроса. Любая ошибка возвращается в едином виде — общее сообщение и словарь «поле — сообщение», по которому web-клиент подсвечивает поля формы.

Таблица 3.1 – REST-интерфейс микросервиса «Клиенты»

Метод	Адрес	Назначение и коды ответа
GET	/api/clients	список клиентов, отсортированный по фамилии; 200
GET	/api/clients/{id}	данные клиента; 200, 404
POST	/api/clients	добавление; 201, 400 — ошибка валидации, 409 — дубль
PUT	/api/clients/{id}	изменение; 200, 400, 404, 409
DELETE	/api/clients/{id}	удаление; 204, 404
GET	/api/dictionaries	содержимое четырёх справочников для списков формы

4 РЕАЛИЗАЦИЯ

4.1 Структура проекта

Сервис является модулем многомодульного Maven-проекта банка. Код разложен по слоям (листинг 4.1); общие для всех web-клиентов стили, маски ввода и обёртка над fetch вынесены в модуль ui-kit.

Листинг 4.1 – Структура модуля client-service

```
src/main/java/by/bsuir/bank/client/
  domain/      Client, справочники City, MaritalStatus, Citizenship, Disability
  repository/   репозитории Spring Data JPA
  dto/          ClientRequest (форма с правилами валидации), ClientResponse, ApiError
  validation/   Masks – маски полей, Dates и @DateText – строгая проверка дат
  service/      ClientService – уникальность, справочники, сохранение
  web/          ClientController, DictionaryController, ApiExceptionHandler
src/main/resources/
  schema.sql    схема данных          data.sql    справочники и тестовые клиенты
  static/       index.html, app.js – web-клиент на Vue 3
src/test/java/  модульные (*Test), интеграционные и функциональные (*IT) тесты
```

4.2 Web-клиент

Web-клиент — одностраничное приложение на Vue 3 с двумя формами. Форма «Список клиентов» запрашивает GET /api/clients и показывает таблицу; сортировку по фамилии выполняет сервис, чтобы порядок был одинаковым для любого потребителя REST. Форма клиента используется и для добавления, и для редактирования; удаление доступно из списка и из формы.

Форма построена по описанию полей: для каждого поля в одном месте заданы подпись, тип, обязательность, маска ввода и функция проверки, а разметка и валидация генерируются по этому описанию. Добавление поля сводится к одной строке, и расхождение между тем, что нарисовано, и тем, что проверяется, становится невозможным. Маска (# — цифра, A — латинская буква) применяется при каждом нажатии клавиши: недопустимые символы отбрасываются, разделители расставляются автоматически, поэтому в поле даты физически нельзя набрать текст.

4.3 Двойная проверка маски и обязательности

Проверка значений выполняется дважды. Первичная проверка в браузере нужна для удобства: пользователь сразу видит сообщения у полей и не ждёт ответа сервера. Но она не защищает базу — запрос можно отправить в обход формы, поэтому сервис повторяет все проверки, и именно он является источником истины.

Первичная проверка на клиенте показана в листинге 4.2. Полужирным выделен фрагмент, который для каждого поля проверяет обязательность

(признак `required`) и соответствие маске (функция `check` с регулярным выражением).

Листинг 4.2 – Первичная проверка в web-клиенте (`static/app.js`)

```
const SECTIONS = [
  ...
  { title: 'Паспортные данные', fields: [
    { name: 'passportSeries', label: 'Серия паспорта', required: true, mask: 'AA', placeholder: 'MP',
      check: v => /^[A-Z]{2}$/.test(v) || 'Серия – две заглавные латинские буквы' },
    { name: 'passportNumber', label: '№ паспорта', required: true, mask: '#####',
      placeholder: '1234567',
      check: v => /\d{7}$/.test(v) || 'Номер паспорта – семь цифр' },
    { name: 'identificationNumber', label: 'Идентификационный номер', required: true,
      mask: '#####A###AA#', placeholder: '3140301A001PB5',
      check: v => /\d{7}[A-Z]\d{3}[A-Z]{2}\d$/.test(v) || 'Формат: 7 цифр, буква, 3 цифры, 2 буквы, цифра' },
    ...
  ]
};

/* Проверка формы перед отправкой: возвращает { имяПоля: сообщение }. Пустой объект – ошибок нет. */
function validate(form) {
  const errors = {};
  for (const field of FIELDS) {
    const raw = form[field.name];
    const value = typeof raw === 'string' ? raw.trim() : raw;
    const empty = value === null || value === undefined || value === '';
    if (empty) {
      if (field.required) errors[field.name] = REQUIRED;
      continue;
    }
    const result = field.check ? field.check(value, form) : true;
    if (result !== true) errors[field.name] = result;
  }
  return errors;
}
```

Проверка в сервисе (листинг 4.3) описана декларативно аннотациями Bean Validation на записи `ClientRequest`; контроллер помечает параметр аннотацией `@Valid`, и до бизнес-логики запрос с ошибками не доходит. Полужирным выделены ограничения обязательности (`@NotBlank`, `@NotNull`) и маски (`@Pattern`, `@DateText`). Регулярные выражения масок собраны в классе `Masks` и по содержанию совпадают с выражениями web-клиента.

Листинг 4.3 – Серверная проверка: запись `ClientRequest` и маски `Masks`

```
public record ClientRequest(

    @NotBlank(message = REQUIRED) @Size(max = 60, message = TOO_LONG)
    @Pattern(regexp = Masks.NAME, message = LETTERS_ONLY)
    String lastName,
    ...
    @NotBlank(message = REQUIRED) @DateText
    String birthDate,

    @NotBlank(message = REQUIRED)
```

```

        @Pattern(regexp = Masks.PASSPORT_SERIES, message = "Серия – две заглавные латинские буквы")
        String passportSeries,

        @NotBlank(message = REQUIRED)
        @Pattern(regexp = Masks.PASSPORT_NUMBER, message = "Номер паспорта – семь цифр")
        String passportNumber,

        ...

        @NotBlank(message = REQUIRED)
        @Pattern(regexp = Masks.IDENTIFICATION_NUMBER, message = "Формат: 7 цифр, буква, 3 цифры, 2 буквы, цифра")
        String identificationNumber,

        ...

        @Pattern(regexp = Masks.MOBILE_PHONE, message = "Формат: +375 (29) 314-15-92")
        String mobilePhone,

        @NotNull(message = REQUIRED)
        Long residenceCityId,

        ...
    ) { ... }

    public final class Masks {
        public static final String NAME = "^[A-Яа-яЁёА-Za-z]+(?:[:-' ][A-Яа-яЁёА-Za-z]+)*$";
        public static final String PASSPORT_SERIES = "^[A-Z]{2}$";
        public static final String PASSPORT_NUMBER = "\\d{7}$";
        public static final String IDENTIFICATION_NUMBER = "\\d{7}[A-Z]\\d{3}[A-Z]{2}\\d$";
        public static final String HOME_PHONE = "^$|^\\d{3}-\\d{2}-\\d{2}$";
        public static final String MOBILE_PHONE = "^$|^\\+375 \\((25|29|33|44)\\) \\d{3}-\\d{2}-\\d{2}$";
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public ClientResponse create(@Valid @RequestBody ClientRequest request) {
        return service.create(request);
    }
}

```

Три детали серверной проверки заслуживают отдельного пояснения.

Пробел вместо имени. Компактный конструктор записи отбрасывает пробелы по краям всех строк ещё до проверок, поэтому строка из одного пробела превращается в пустую и отклоняется как незаполненное обязательное поле.

Несуществующая дата. Стандартный разбор даты в Java по умолчанию работает в режиме SMART и молча превращает 31 февраля в 28-е. Для строгой проверки формат создаётся с `ResolverStyle.STRICT` (листинг 4.4): и 31.02.2015, и 29.02.2015 отвергаются, а 29.02.2016 принимается. Собственное ограничение `@DateText` дополнительно требует, чтобы дата лежала между 01.01.1900 и сегодняшним днём.

Листинг 4.4 – Строгий разбор даты (validation/Dates.java)

```

public static final DateTimeFormatter FORMAT =
    DateTimeFormatter.ofPattern("dd.MM.uuuu").withResolverStyle(ResolverStyle.STRICT);

public static Optional<LocalDate> parse(String text) {
    if (text == null) {

```

```

        return Optional.empty();
    }
    try {
        return Optional.of(LocalDate.parse(text.trim(), FORMAT));
    } catch (DateTimeParseException e) {
        return Optional.empty();
    }
}

```

Необязательные поля. Для телефонов и e-mail маска допускает пустую строку (^\$|...), а сервис сохраняет пустое значение как NULL: запись клиента только с обязательными полями выполняется без ошибок.

4.4 Уникальность паспорта и идентификационного номера

Маска не может обнаружить повторный ввод того же клиента — это правило проверяется в сервисе обращением к базе (листинг 4.5). При редактировании собственная запись клиента из сравнения исключается, иначе его нельзя было бы сохранить без смены паспорта. Нарушение возвращается с кодом 409 и сообщениями у полей паспорта и идентификационного номера. Если два одинаковых запроса придут одновременно и оба пройдут проверку, второй будет остановлен ограничением уникальности в базе; обработчик ошибок превращает `DataIntegrityViolationException` в тот же ответ 409.

Листинг 4.5 – Проверка уникальности (service/ClientService.java)

```

private void checkUnique(ClientRequest request, Long selfId) {
    Map<String, String> errors = new LinkedHashMap<>();
    clients.findByPassportSeriesAndPassportNumber(request.passportSeries(), request.
        passportNumber())
        .filter(other -> !other.getId().equals(selfId))
        .ifPresent(other -> {
            errors.put("passportSeries", DUPLICATE_PASSPORT);
            errors.put("passportNumber", DUPLICATE_PASSPORT);
        });
    clients.findByIdentificationNumber(request.identificationNumber())
        .filter(other -> !other.getId().equals(selfId))
        .ifPresent(other -> errors.put("identificationNumber", DUPLICATE_IDENTIFICATION));
    if (!errors.isEmpty()) {
        throw new FieldErrorException(HttpStatus.CONFLICT, "Такой клиент уже есть в базе",
            errors);
    }
}

```

5 ТЕСТИРОВАНИЕ

5.1 Уровни тестов

Тесты образуют пирамиду из трёх уровней (таблица 5.1). Модульные тесты запускаются на фазе `test` и не поднимают Spring; интеграционные и функциональные (имена классов оканчиваются на `IT`) выполняются плагином Failsafe на фазе `verify`. Все уровни используют базу H2 в памяти; у каждого класса тестов своя база, поэтому тесты не влияют друг на друга.

Таблица 5.1 – Тесты микросервиса «Клиенты»

Класс тестов	Тестов	Что проверяется
<i>Модульные — без Spring и базы данных</i>		
DateTextValidatorTest	16	существующие и несуществующие даты
ClientRequestValidationTest	49	маски и обязательность каждого поля
ClientServiceTest	8	уникальность и справочники на заглушках Mockito
<i>Интеграционные — сервис целиком и H2</i>		
ClientApiIT	39	REST через MockMvc, серверная валидация, состояние таблицы
<i>Функциональные — браузер через WebDriver</i>		
ClientUiIT	36	сценарии программы защиты в web-клиенте
Всего	148	

5.2 Программа защиты в виде автоматических тестов

Функциональные тесты повторяют пункты, по которым проводится защита работы (таблица 5.2). Под данными «Иванов Иван Иванович» в задании понимаются данные студента; в тестах пунктов 2 и 3 новый клиент вводится с паспортом и идентификационным номером клиента Лебедевич из начального заполнения базы. Каждый тест после действий в браузере выполняет SQL-запрос к таблице `client` — проверку «прямо в базе».

Таблица 5.2 – Соответствие пунктов защиты и тестов

Пункт	Сценарий и ожидаемый результат	Тест ClientUiIT
1	те же данные ещё раз — ошибка, в базе один клиент	sameClientTwice
2	другой клиент с тем же паспортом — ошибка	anotherClientWithSamePassport
3	другой клиент с тем же идентификационным номером — ошибка	...SameIdentificationNumber
4	фамилия «1234» — ошибка, фамилии 1234 в базе нет	digitsInsteadOfLastName
5	пробел вместо имени — ошибка, клиент не записан	spaceInsteadOfFirstName
6	не дата, 31.02.2015, 29.02.2015; все поля с маской	birthDateIsNotDate, maskedFields
7	нет одного обязательного поля (17 вариантов) — ошибка	requiredFieldIsMissing
8	только обязательные поля — клиент записан	onlyRequiredFields
9	двойная проверка на клиенте и на сервере	листинги 4.2, 4.3

Поскольку web-клиент не пропускает некорректные данные до сервера, серверная валидация проверяется отдельно: интеграционные тесты ClientApiIT отправляют те же неверные данные прямо в REST, минуя браузер, и убеждаются, что сервис отвечает кодом 400 или 409 и ничего не пишет в базу. Пример теста каждого уровня приведён в листингах 5.1 и 5.2.

Листинг 5.1 – Интеграционный тест пункта 6 (ClientApiIT)

```
@ParameterizedTest(name = "дата рождения «{0}»")
@ValueSource(strings = {"не дата", "31.02.2015", "29.02.2015"})
@DisplayName("п.6 Текст, не являющийся датой, – ошибка")
void birthDateIsNotDate(String birthDate) throws Exception {
    create(TestClients.with("birthDate", birthDate))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.fields.birthDate").exists());

    assertThat(count("last_name = 'Иванов'")).isZero();
}

/** Проверка «прямо в базе»: SQL-запрос к таблице client в обход сервиса и репозиториев. */
private int count(String where) {
    return jdbc.queryForObject("select count(*) from client where " + where, Integer.class);
}
```

Листинг 5.2 – Функциональный тест пункта 2 через WebDriver (ClientUiIT)

```
@Test
@DisplayName("п.2 Другой клиент с паспортом студента – сообщение об ошибке")
void anotherClientWithSamePassport() {
    Map<String, Object> other = TestClients.valid();
    other.put("passportSeries", "MP");
    other.put("passportNumber", "3141592"); // паспорт клиента Лебедевич из начальных
    // данных
}
```

```

fill(other);
save();

assertThat(error("passportSeries")).isEqualTo("Клиент с таким паспортом уже
zaregistririvan");
assertThat(count("passport_series = 'MP' and passport_number = '3141592'")).isEqualTo(1);
assertThat(count("last_name = 'Иванов'")).isZero();
}

```

5.3 Результаты прогона

Команда `./mvnw -pl client-service -am verify` собирает сервис и выполняет все 148 тестов; функциональные тесты запускают Chrome без окна, драйвер для него загружает Selenium Manager. Итог прогона приведён в листинге 5.3.

Листинг 5.3 – Результат выполнения тестов

```

$ ./mvnw -pl client-service -am verify

Tests run: 16, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.061 s -- in by.bsuir.bank.client.DateTextValidatorTest
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.576 s -- in by.bsuir.bank.client.ClientServiceTest
Tests run: 49, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.378 s -- in by.bsuir.bank.client.ClientRequestValidationTest
Tests run: 39, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.135 s -- in by.bsuir.bank.client.ClientApiIT
Tests run: 36, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 39.92 s -- in by.bsuir.bank.client.ClientUiIT

Tests run: 148, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

```

6 РЕЗУЛЬТАТЫ РАБОТЫ

После запуска сервис доступен по адресу `http://localhost:8081`. Форма «Список клиентов» с начальным заполнением базы показана на рисунке 6.1: шесть клиентов отсортированы по фамилии.

Банк «Решение» учебная АБС **Клиенты** Счета Депозиты Кредиты Банкомат

Список клиентов
Всего клиентов: 6. Сортировка по фамилии. Добавить клиента

ФИО	ДАТА РОЖДЕНИЯ	ПАСПОРТ	ИДЕНТ. НОМЕР	ГОРОД ПРОЖИВАНИЯ	МОБИЛЬНЫЙ ТЕЛЕФОН	ДОХОД, ВУН	
Абрамович Елена Сергеевна	28.02.1987	КН 1414213	4280287K003PB2	Гродно	+375 (44) 141-42-13		Изменить Удалить
Жуков Дмитрий Александрович	09.09.1979	ВМ 1732050	3090979E045PB6	Гомель	+375 (25) 173-20-50	5 600,00	Изменить Удалить
Ковальчук Игорь Петрович	23.07.1958	АВ 1618033	3230758C027PB8	Брест		1 180,00	Изменить Удалить
Лебедев Артем Владимирович	14.03.2001	МР 3141592	3140301A001PB5	Минск	+375 (29) 314-15-92	4 200,00	Изменить Удалить
Савицкая Марина Олеговна	05.11.1994	МС 2718281	4051194B014PB1	Минск	+375 (33) 271-82-81	3 100,50	Изменить Удалить
Шевчук Ольга Николаевна	01.12.1951	ВМ 2236067	4011251H019PB3	Витебск		920,40	Изменить Удалить

Рисунок 6.1 – Форма «Список клиентов»

Форма добавления клиента, заполненная корректными данными, приведена на рисунке 6.2. Обязательные поля отмечены звездочкой, списки заполнены из справочников, значения полей с маской набраны с автоматической расстановкой разделителей.

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Новый клиент

Поля, отмеченные *, обязательны для заполнения.

ЛИЧНЫЕ ДАННЫЕ

Фамилия *

Иванов

Имя *

Иван

Отчество *

Иванович

Дата рождения *

17.05.1990

Пол *

☒ Мужской ☐ Женский

Место рождения *

г. Минск

ПАСПОРТНЫЕ ДАННЫЕ

Серия паспорта *

MP

№ паспорта *

7654321

Идентификационный номер *

3170590A077PB4

Кем выдан *

Фрунзенское РУВД г. Минска

Дата выдачи *

20.06.2018

ПРОЖИВАНИЕ И КОНТАКТЫ

Город фактического проживания *

Минск

Адрес фактического проживания *

ул. Притыцкого, д. 29, кв. 3

Город прописки *

Борисов

Телефон домашний

201-45-67

Телефон мобильный

+375 (29) 765-43-21

E-mail

ivanov@example.by

СОЦИАЛЬНЫЙ СТАТУС

Семейное положение *

Женат / замужем

Гражданство *

Республика Беларусь

Инвалидность *

Нет

Ежемесячный доход, BYN

2500.50

Пенсионер

☐ да

Сохранить

Отмена

Рисунок 6.2 – Форма добавления клиента

На рисунке 6.3 показана реакция первичной проверки на ошибки из программы защиты: цифры в фамилии, пробел вместо имени, дата 31.02.2015, неполный номер паспорта, мобильный телефон с недопустимым кодом и неполный e-mail. Запрос на сервер при этом не отправляется.

Банк «Решение» учебная АБС Клиенты Счета Депозиты Кредиты Банкомат

Новый клиент

Поля, отмеченные *, обязательны для заполнения.

Форма заполнена с ошибками — исправьте отмеченные поля

ЛИЧНЫЕ ДАННЫЕ

Фамилия *	Имя *	Отчество *
1234		Иванович
Допустимы только буквы	Обязательное поле	
Дата рождения *	Пол *	Место рождения *
31.02.2015	☒ Мужской ☐ Женский	г. Минск
Такой даты не существует, формат ДД.ММ.ГГГГ		

ПАСПОРТНЫЕ ДАННЫЕ

Серия паспорта *	№ паспорта *	Идентификационный номер *
MP	12345	3170590A077PB4
	Номер паспорта — семь цифр	
Кем выдан *	Дата выдачи *	
Фрунзенское РУВД г. Минска	20.06.2018	

ПРОЖИВАНИЕ И КОНТАКТЫ

Город фактического проживания *	Адрес фактического проживания *	
Минск	ул. Притыцкого, д. 29, кв. 3	
Город прописки *	Телефон домашний	Телефон мобильный
Борисов	201-45-67	+375 (17) 765-43-21
		Формат: +375 (29) 314-15-92, код 25, 29, 33 или 44
E-mail		
ivanov@		
Некорректный e-mail		

СОЦИАЛЬНЫЙ СТАТУС

Семейное положение *	Гражданство *	Инвалидность *
Женат / замужем	Республика Беларусь	Нет
Ежемесячный доход, BYN	Пенсионер	
2500.50	☐ да	

Сохранить Отмена

Рисунок 6.3 – Сообщения первичной проверки в браузере

Ошибку повторного ввода обнаруживает только сервер. На рисунке 6.4 новый клиент введён с паспортом и идентификационным номером, которые уже принадлежат клиенту из базы: сервис ответил кодом 409, а web-клиент показал его сообщения у соответствующих полей.

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Новый клиент

Поля, отмеченные *, обязательны для заполнения.

Такой клиент уже есть в базе

ЛИЧНЫЕ ДАННЫЕ

Фамилия *

Иванов

Имя *

Иван

Отчество *

Иванович

Дата рождения *

17.05.1990

Пол *

☒ Мужской ☐ Женский

Место рождения *

г. Минск

ПАСПОРТНЫЕ ДАННЫЕ

Серия паспорта *

MP

Клиент с таким паспортом уже зарегистрирован

№ паспорта *

3141592

Клиент с таким паспортом уже зарегистрирован

Идентификационный номер *

3140301A001PB5

Клиент с таким идентификационным номером уже зарегистрирован

Кем выдан *

Фрунзенское РУВД г. Минска

Дата выдачи *

20.06.2018

ПРОЖИВАНИЕ И КОНТАКТЫ

Город фактического проживания *

Минск

Адрес фактического проживания *

ул. Притыцкого, д. 29, кв. 3

Город прописки *

Борисов

Телефон домашний

201-45-67

Телефон мобильный

+375 (29) 765-43-21

E-mail

ivanov@example.by

СОЦИАЛЬНЫЙ СТАТУС

Семейное положение *

Женат / замужем

Гражданство *

Республика Беларусь

Инвалидность *

Нет

Ежемесячный доход, BYN

2500.50

Пенсионер

☐ да

Сохранить

Отмена

Рисунок 6.4 – Сообщения сервиса о нарушении уникальности

После исправления данных клиент сохраняется и появляется в списке на своём месте по алфавиту (рисунок 6.5).

Банк «Решение» учебная ABC

КлиентыСчетаДепозитыКредитыБанкомат

Список клиентов

Всего клиентов: 7. Сортировка по фамилии.

Добавить клиента

Клиент добавлен

ФИО	ДАТА РОЖДЕНИЯ	ПАСПОРТ	ИДЕНТ. НОМЕР	ГОРОД ПРОЖИВАНИЯ	МОБИЛЬНЫЙ ТЕЛЕФОН	ДОХОД, BYN	
Абрамович Елена Сергеевна	28.02.1987	КН 1414213	4280287K003PB2	Гродно	+375 (44) 141-42-13		ИзменитьУдалить
Жуков Дмитрий Александрович	09.09.1979	ВМ 1732050	3090979E045PB6	Гомель	+375 (25) 173-20-50	5 600,00	ИзменитьУдалить
Иванов Иван Иванович	17.05.1990	МР 7654321	3170590A077PB4	Минск	+375 (29) 765-43-21	2 500,50	ИзменитьУдалить
Ковальчук Игорь Петрович	23.07.1958	АВ 1618033	3230758C027PB8	Брест		1 180,00	ИзменитьУдалить
Лебедев Артем Владимирович	14.03.2001	МР 3141592	3140301A001PB5	Минск	+375 (29) 314-15-92	4 200,00	ИзменитьУдалить
Савицкая Марина Олеговна	05.11.1994	МС 2718281	4051194B014PB1	Минск	+375 (33) 271-82-81	3 100,50	ИзменитьУдалить
Шевчук Ольга Николаевна	01.12.1951	ВМ 2236067	4011251H019PB3	Витебск		920,40	ИзменитьУдалить

Рисунок 6.5 – Список после добавления клиента

Состояние таблицы можно проверить напрямую в консоли H2 по адресу /h2-console (JDBC URL jdbc:h2:mem:clients). На рисунке 6.6 видно, что после всех попыток в таблице семь записей и нет ни одинаковых паспортов, ни одинаковых идентификационных номеров, ни клиента с фамилией «1234».

jdbc:h2:mem:clients

CITIZENSHIPCITYCLIENTDISABILITYMARITAL_STATUSINFORMATION_SCHEMAUsersH2 2.3.232 (2024-08-11)

RunRun SelectedAuto completeClearSQL statement:

SELECT ID, LAST_NAME, FIRST_NAME, PASSPORT_SERIES, PASSPORT_NUMBER, IDENTIFICATION_NUMBER FROM CLIENT ORDER BY LAST_NAME

SELECT ID, LAST_NAME, FIRST_NAME, PASSPORT_SERIES, PASSPORT_NUMBER, IDENTIFICATION_NUMBER FROM CLIENT ORDER BY LAST_NAME;

ID	LAST_NAME	FIRST_NAME	PASSPORT_SERIES	PASSPORT_NUMBER	IDENTIFICATION_NUMBER
4	Абрамович	Елена	КН	1414213	4280287K003PB2
5	Жуков	Дмитрий	ВМ	1732050	3090979E045PB6
7	Иванов	Иван	МР	7654321	3170590A077PB4
3	Ковальчук	Игорь	АВ	1618033	3230758C027PB8
1	Лебедев	Артем	МР	3141592	3140301A001PB5
2	Савицкая	Марина	МС	2718281	4051194B014PB1
6	Шевчук	Ольга	ВМ	2236067	4011251H019PB3

(7 rows, 1 ms)

Edit

Рисунок 6.6 – Таблица client в консоли H2

ЗАКЛЮЧЕНИЕ

В ходе работы разработан микросервис «Клиенты» на Java и Spring Boot с базой данных H2 в памяти, REST-интерфейсом и web-клиентом на Vue 3. Схема данных содержит таблицу клиентов и четыре справочника для полей типа «Список»; база создаётся и заполняется автоматически при старте приложения.

Корректность данных обеспечивается на трёх уровнях. Web-клиент применяет маски ввода и проверяет форму до отправки, сервис повторяет проверку масок и обязательности средствами Bean Validation и контролирует уникальность паспорта и идентификационного номера, а база данных хранит ограничения уникальности и ссылочной целостности. Такое разделение соответствует практике реальных банковских систем: проверка в браузере служит удобству пользователя, а гарантии целостности даёт только сервер.

Работоспособность подтверждена 148 автоматическими тестами: 73 модульными, 39 интеграционными и 36 функциональными, которые через WebDriver проходят все пункты программы защиты и сверяют результат с содержимым таблицы в базе.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Уоллс, К. Spring в действии / К. Уоллс. — 6-е изд. — М.: ДМК Пресс, 2022. — 544 с.
2. Spring Boot Reference Documentation [Электронный ресурс]. — Режим доступа: <https://docs.spring.io/spring-boot/>. — Дата доступа: 01.10.2026.
3. Jakarta Bean Validation 3.0 Specification [Электронный ресурс]. — Режим доступа: <https://jakarta.ee/specifications/bean-validation/3.0/>. — Дата доступа: 01.10.2026.
4. Selenium WebDriver Documentation [Электронный ресурс]. — Режим доступа: <https://www.selenium.dev/documentation/webdriver/>. — Дата доступа: 01.10.2026.
5. Vue.js Guide [Электронный ресурс]. — Режим доступа: <https://vuejs.org/guide/>. — Дата доступа: 01.10.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код микросервиса «Клиенты»

Сборка проекта

Листинг А.1 – pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.16</version>
    <relativePath/>
  </parent>

  <groupId>by.bsuir.bank</groupId>
  <artifactId>bank</artifactId>
  <version>1.0.0</version>
  <packaging>pom</packaging>
  <name>bank</name>
  <description>Информационная система условного банка: микросервисы на Java + Spring</description>

  <modules>
    <module>ui-kit</module>
    <module>bank-common</module>
    <module>client-service</module>
    <module>account-service</module>
    <module>deposit-service</module>
    <module>credit-service</module>
    <module>atm-service</module>
  </modules>

  <properties>
    <java.version>17</java.version>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <vue.version>3.5.13</vue.version>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>by.bsuir.bank</groupId>
        <artifactId>ui-kit</artifactId>
        <version>${project.version}</version>
      </dependency>
      <dependency>
        <groupId>by.bsuir.bank</groupId>
        <artifactId>bank-common</artifactId>
        <version>${project.version}</version>
      </dependency>
      <dependency>
        <groupId>org.webjars.npm</groupId>
        <artifactId>vue</artifactId>
        <version>${vue.version}</version>
      </dependency>
    </dependencies>
  </dependencyManagement>

  <build>
    <pluginManagement>
      <plugins>
        <!-- *Test – модульные тесты (фаза test), *IT – интеграционные и функциональные (фаза verify) -->
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-failsafe-plugin</artifactId>
          <executions>
            <execution>
```

```

        <goals>
            <goal>integration-test</goal>
            <goal>verify</goal>
        </goals>
    </execution>
</executions>
<configuration>
    <classesDirectory>${project.build.outputDirectory}</classesDirectory>
</configuration>
</plugin>
</plugins>
</pluginManagement>
</build>
</project>

```

Общие ресурсы web-клиентов (ui-kit)

Листинг A.2 – ui-kit/pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>by.bsuir.bank</groupId>
        <artifactId>bank</artifactId>
        <version>1.0.0</version>
    </parent>

    <artifactId>ui-kit</artifactId>
    <name>ui-kit</name>
    <description>Общие статические ресурсы web-клиентов: стили, маски ввода, обёртка над fetch</description>

    <dependencies>
        <!-- Vue 3 раздётся самим сервисом из webjars, без CDN и без сборки фронтенда -->
        <dependency>
            <groupId>org.webjars.npm</groupId>
            <artifactId>vue</artifactId>
        </dependency>
        <dependency>
            <groupId>org.webjars</groupId>
            <artifactId>webjars-locator-lite</artifactId>
        </dependency>
    </dependencies>
</project>

```

Листинг A.3 – ui-kit/src/main/resources/META-INF/resources/ui/bank.css

```

/* Общее оформление web-клиентов банка */
:root {
    --navy: #0b2545;
    --blue: #1d5fa8;
    --blue-soft: #e8f0fa;
    --ink: #1c2733;
    --muted: #66758a;
    --line: #d9e0ea;
    --bg: #f3f5f9;
    --ok: #1f8a4c;
    --ok-soft: #e4f5ea;
    --bad: #c0392b;
    --bad-soft: #fdecea;
    --radius: 8px;
}

* { box-sizing: border-box; }

body {
    margin: 0;
    background: var(--bg);
    color: var(--ink);
    font: 15px/1.45 -apple-system, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif;
}

```

```

[v-cloak] { display: none; }

.topbar {
  display: flex;
  align-items: center;
  gap: 32px;
  padding: 0 28px;
  height: 56px;
  background: var(--navy);
  color: #fff;
}
.topbar .brand { font-weight: 700; font-size: 17px; letter-spacing: .2px; }
.topbar .brand span { font-weight: 400; opacity: .6; margin-left: 8px; font-size: 13px; }
.topbar nav { display: flex; gap: 4px; }
.topbar nav a {
  color: #c9d6e8;
  text-decoration: none;
  padding: 8px 14px;
  border-radius: 6px;
}
.topbar nav a:hover { background: rgba(255, 255, 255, .08); color: #fff; }
.topbar nav a.active { background: var(--blue); color: #fff; }

main { max-width: 1240px; margin: 24px auto 48px; padding: 0 24px; }

h1 { font-size: 22px; margin: 0 0 4px; }
h2 { font-size: 17px; margin: 0 0 12px; }
.subtitle { color: var(--muted); margin: 0 0 18px; }

.card {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: var(--radius);
  padding: 20px 22px;
  margin-bottom: 18px;
}

.toolbar { display: flex; align-items: center; justify-content: space-between; gap: 12px; margin-bottom: 14px; }
.toolbar .spacer { flex: 1; }

button, .btn {
  font: inherit;
  cursor: pointer;
  border: 1px solid var(--line);
  background: #fff;
  color: var(--ink);
  padding: 7px 14px;
  border-radius: 6px;
}
button:hover { border-color: var(--blue); }
button.primary { background: var(--blue); border-color: var(--blue); color: #fff; }
button.primary:hover { background: #174f8d; }
button.danger { color: var(--bad); }
button.danger:hover { border-color: var(--bad); background: var(--bad-soft); }
button.small { padding: 3px 9px; font-size: 13px; }
button.disabled { opacity: .5; cursor: default; }

table { width: 100%; border-collapse: collapse; }
th, td { text-align: left; padding: 8px 10px; border-bottom: 1px solid var(--line); vertical-align: top; }
th { font-size: 12px; text-align: right; font-variant-numeric: tabular-nums; white-space: nowrap; }
tbody tr:hover { background: #f8fafd; }
td.num, th.num { text-align: right; font-variant-numeric: tabular-nums; white-space: nowrap; }
td.mono { font-family: "SF Mono", Menlo, Consolas, monospace; font-size: 13px; white-space: nowrap; }
td.actions { white-space: nowrap; text-align: right; }
tfoot td { font-weight: 600; border-bottom: none; }
.empty { color: var(--muted); padding: 18px 0; text-align: center; }

.form-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 14px 20px; }
.form-grid .wide { grid-column: span 2; }
.form-grid .full { grid-column: 1 / -1; }
fieldset { border: 1px solid var(--line); border-radius: var(--radius); margin: 0 0 16px; padding: 14px 18px 18px; }
legend { padding: 0 8px; color: var(--muted); font-size: 13px; text-transform: uppercase; letter-spacing: .4px; }

.field label { display: block; font-size: 13px; color: var(--muted); margin-bottom: 4px; }
.field label .req { color: var(--bad); }
.field input[type=text], .field select {
  width: 100%;
  font: inherit;
  padding: 7px 10px;
  border: 1px solid var(--line);
  border-radius: 6px;
  background: #fff;
}

```

```

}
.field input:focus, .field select:focus { outline: 2px solid var(--blue-soft); border-color: var(--blue); }
.field input[readonly] { background: #f3f5f9; color: var(--muted); }
.field.invalid input, .field.invalid select { border-color: var(--bad); background: var(--bad-soft); }
.field.error { color: var(--bad); font-size: 12.5px; margin-top: 3px; }
.field.hint { color: var(--muted); font-size: 12.5px; margin-top: 3px; }
.field.choices { display: flex; gap: 18px; padding: 7px 0; }
.field.choices label { display: inline-flex; align-items: center; gap: 6px; color: var(--ink); font-size: 15px; margin: 0; }

.form-actions { display: flex; gap: 10px; margin-top: 6px; }

.alert { padding: 10px 14px; border-radius: 6px; margin-bottom: 14px; }
.alert.error { background: var(--bad-soft); color: var(--bad); border: 1px solid #f3c2bc; }
.alert.success { background: var(--ok-soft); color: var(--ok); border: 1px solid #b9e2c7; }
.alert.info { background: var(--blue-soft); color: var(--navy); border: 1px solid #c5d8ef; }

.badge { display: inline-block; padding: 1px 8px; border-radius: 10px; font-size: 12px; background: var(--blue-soft); color: var(--navy); }
.badge.ok { background: var(--ok-soft); color: var(--ok); }
.badge.off { background: #e2eff3; color: var(--muted); }
.badge.bad { background: var(--bad-soft); color: var(--bad); }

.stats { display: grid; grid-template-columns: repeat(4, 1fr); gap: 14px; margin-bottom: 18px; }
.stat { background: #fff; border: 1px solid var(--line); border-radius: var(--radius); padding: 14px 18px; }
.stat.label { color: var(--muted); font-size: 12px; text-transform: uppercase; letter-spacing: .4px; }
.stat.value { font-size: 20px; font-weight: 600; margin-top: 2px; font-variant-numeric: tabular-nums; }

.muted { color: var(--muted); }
.nowrap { white-space: nowrap; }
.right { text-align: right; }

.bankday { display: flex; align-items: center; gap: 8px; background: #fff; border: 1px solid var(--line); border-radius: var(--radius); padding: 8px 12px; }
.bankday.label { color: var(--muted); font-size: 12px; text-transform: uppercase; letter-spacing: .4px; }
.bankday.b { font-size: 16px; margin-right: 8px; font-variant-numeric: tabular-nums; }
.bankday input { width: 56px; font: inherit; padding: 6px 8px; border: 1px solid var(--line); border-radius: 6px; text-align: right; }

.details { display: grid; grid-template-columns: repeat(4, 1fr); gap: 12px 20px; }
.details.label { color: var(--muted); font-size: 12px; text-transform: uppercase; letter-spacing: .4px; }
.details.value { font-weight: 600; }
a.link { color: var(--blue); cursor: pointer; text-decoration: none; }
a.link:hover { text-decoration: underline; }

```

Листинг А.4 – ui-kit/src/main/resources/META-INF/resources/ui/bank.js

```

/* Общие помощники web-клиентов банка: REST-вызовы, маски ввода, форматирование. */
const Bank = {

  /* Адреса web-клиентов остальных микросервисов для общей навигации. */
  services: [
    { key: 'clients', title: 'Клиенты', port: 8081 },
    { key: 'accounts', title: 'Счета', port: 8082 },
    { key: 'deposits', title: 'Депозиты', port: 8083 },
    { key: 'credits', title: 'Кредиты', port: 8084 },
    { key: 'atm', title: 'Банкомат', port: 8085 },
  ],

  /* REST-вызов: при ошибке бросает { status, message, fields } из тела ответа сервиса. */
  async api(method, url, body) {
    const options = { method, headers: { 'Accept': 'application/json' } };
    if (body !== undefined) {
      options.headers['Content-Type'] = 'application/json';
      options.body = JSON.stringify(body);
    }
    let response;
    try {
      response = await fetch(url, options);
    } catch (e) {
      throw { status: 0, message: 'Сервис недоступен', fields: {} };
    }
    const text = await response.text();
    const data = text ? JSON.parse(text) : null;
    if (!response.ok) {
      throw {
        status: response.status,
        message: (data && data.message) || 'Ошибка ' + response.status,
        fields: (data && data.fields) || {},
      };
    }
  }
}

```

```

    return data;
  },

  /* Маска ввода: # — цифра, A — латинская буква (в верхнем регистре), остальное — литералы. */
  mask(value, pattern) {
    const raw = String(value ?? '');
    let out = '';
    let i = 0;
    for (const p of pattern) {
      if (i >= raw.length) break;
      if (p === '#' || p === 'A') {
        const allowed = p === '#' ? /\d/ : /[a-zA-Z]/;
        while (i < raw.length && !allowed.test(raw[i])) i++;
        if (i >= raw.length) break;
        out += raw[i++].toUpperCase();
      } else {
        out += p;
        if (raw[i] === p) i++;
      }
    }
    return out;
  },

  /* Директива v-mask="'###.##.####'" — применяет маску при каждом вводе. */
  maskDirective: {
    mounted(el, binding) {
      el.addEventListener('input', () => {
        const masked = Bank.mask(el.value, binding.value);
        if (masked !== el.value) {
          el.value = masked;
          el.dispatchEvent(new Event('input'));
        }
      });
    },
  },

  /* Строгий разбор даты ДД.ММ.ГГГГ: 31.02.2015 и 29.02.2015 датами не считаются. */
  parseDate(text) {
    const m = /^(\d{2})\.(\d{2})\.(\d{4})$/ .exec(text || '');
    if (!m) return null;
    const [day, month, year] = [Number(m[1]), Number(m[2]), Number(m[3])];
    const date = new Date(year, month - 1, day);
    const same = date.getFullYear() === year && date.getMonth() === month - 1 && date.getDate() === day;
    return same ? date : null;
  },

  money(value) {
    if (value === null || value === undefined || value === '') return '';
    return Number(value).toLocaleString('ru-RU', { minimumFractionDigits: 2, maximumFractionDigits: 2 });
  },

  /* ISO-дата 2026-10-01 -> 01.10.2026 */
  date(iso) {
    return iso ? String(iso).split('-').reverse().join('.') : '';
  },

  /* Панель банковского дня: текущая дата и процедура «Закрытие банковского дня». */
  dayComponent: {
    emits: ['changed', 'failed'],
    data() {
      return { date: null, days: 30, busy: false };
    },
    async mounted() {
      try {
        this.date = (await Bank.api('GET', '/api/bank-day')).date;
      } catch (e) {
        this.$emit('failed', e);
      }
    },
    methods: {
      async close(days) {
        this.busy = true;
        try {
          const result = await Bank.api('POST', '/api/bank-day/close?days=' + days);
          this.date = result.bankDate;
          this.$emit('changed', result);
        } catch (e) {
          this.$emit('failed', e);
        } finally {
          this.busy = false;
        }
      },
    },
  },

```

```

        format: iso => Bank.date(iso),
    },
    template: `
        <div class="bankday">
            <span class="label">Банковский день</span>
            <b id="bank-date">{{ format(date) }}</b>
            <button id="btn-close-day" class="primary" :disabled="busy" @click="close(1)">Закрыть день</button>
            <input id="days" type="text" v-model.number="days" maxlength="4">
            <button id="btn-close-days" :disabled="busy || !(days > 0)" @click="close(days)">Закрыть дней</button>
        </div>`,
    },
},
/* Шапка с переходами между web-клиентами микросервисов. */
navComponent: {
    props: ['active'],
    data() {
        return { services: Bank.services, host: location.protocol + '//' + location.hostname };
    },
    template: `
        <header class="topbar">
            <div class="brand">Банк «Решение» <span>учебная АБС</span></div>
            <nav>
                <a v-for="s in services" :key="s.key" :href="host + ':' + s.port + '/'"
                  :class="{ active: s.key === active }">{{ s.title }}</a>
            </nav>
        </header>`,
    },
},
};

```

Микросервис «Клиенты» (client-service)

Листинг А.5 – client-service/pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>by.bsuir.bank</groupId>
        <artifactId>bank</artifactId>
        <version>1.0.0</version>
    </parent>

    <artifactId>client-service</artifactId>
    <name>client-service</name>
    <description>ЛР1. Микросервис «Клиенты»: ввод и модификация данных о клиентах банка</description>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>by.bsuir.bank</groupId>
            <artifactId>ui-kit</artifactId>
        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>
    </dependencies>

```

```

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-failsafe-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>

```

Листинг А.6 – client-service/src/main/resources/application.yml

```

server:
  port: ${CLIENT_SERVICE_PORT:8081}

spring:
  application:
    name: client-service
  datasource:
    # База в памяти: создаётся и заполняется при каждом старте (schema.sql + data.sql)
    url: jdbc:h2:mem:clients;DB_CLOSE_DELAY=-1
    username: sa
  sql:
    init:
      mode: always
  jpa:
    hibernate:
      ddl-auto: validate
    open-in-view: false
  h2:
    console:
      enabled: true          # http://localhost:8081/h2-console – проверка таблиц «прямо в базе»
  jackson:
    deserialization:
      fail-on-unknown-properties: false

```

Листинг А.7 – client-service/src/main/resources/schema.sql

```

-- Схема данных модуля «Клиенты». Поля типа «Список» вынесены в отдельные справочники.

create table city (
  id bigint generated by default as identity primary key,
  name varchar(100) not null unique
);

create table marital_status (
  id bigint generated by default as identity primary key,
  name varchar(100) not null unique
);

create table citizenship (
  id bigint generated by default as identity primary key,
  name varchar(100) not null unique
);

create table disability (
  id bigint generated by default as identity primary key,
  name varchar(100) not null unique
);

create table client (
  id bigint generated by default as identity primary key,
  last_name varchar(60) not null,

```

```

first_name      varchar(60) not null,
middle_name     varchar(60) not null,
birth_date      date       not null,
sex             varchar(1)  not null check (sex in ('M', 'F')),
passport_series varchar(2)  not null,
passport_number varchar(7)  not null,
issued_by       varchar(200) not null,
issue_date       date       not null,
identification_number varchar(14) not null,
birth_place      varchar(200) not null,
residence_city_id bigint    not null references city,
residence_address varchar(200) not null,
home_phone       varchar(20),
mobile_phone     varchar(20),
email            varchar(100),
registration_city_id bigint    not null references city,
marital_status_id bigint    not null references marital_status,
citizenship_id   bigint    not null references citizenship,
disability_id    bigint    not null references disability,
pensioner        boolean    not null,
monthly_income   decimal(14, 2),
-- последний рубеж защиты от дублей: даже в обход сервиса два одинаковых паспорта не записать
constraint uq_client_passport unique (passport_series, passport_number),
constraint uq_client_identification unique (identification_number)
);

create index ix_client_fio on client (last_name, first_name, middle_name);

```

Листинг А.8 – client-service/src/main/resources/data.sql

```

-- Справочники
insert into city (name) values
    ('Минск'), ('Брест'), ('Витебск'), ('Гомель'), ('Гродно'), ('Могилёв'), ('Барановичи'), ('Борисов');

insert into marital_status (name) values
    ('Холост / не замужем'), ('Женат / замужем'), ('Разведён / разведена'), ('Вдовец / вдова');

insert into citizenship (name) values
    ('Республика Беларусь'), ('Российская Федерация'), ('Республика Казахстан'), ('Украина'), ('Литовская Республика');

insert into disability (name) values
    ('Нет'), ('I группа'), ('II группа'), ('III группа');

-- Тестовые клиенты (паспортные данные вымышленные)
insert into client (last_name, first_name, middle_name, birth_date, sex, passport_series, passport_number,
    issued_by, issue_date, identification_number, birth_place, residence_city_id,
    residence_address, home_phone, mobile_phone, email, registration_city_id,
    marital_status_id, citizenship_id, disability_id, pensioner, monthly_income)
values ('Лебедев', 'Артём', 'Владимирович', date '2001-03-14', 'М', 'МР', '3141592',
    'Советское РУВД г. Минска', date '2019-04-02', '3140301A001PB5', 'г. Минск', 1,
    'ул. Петруся Бровки, д. 6, кв. 12', '293-88-44', '+375 (29) 314-15-92', 'lebedevich@example.by', 1,
    1, 1, 1, false, 4200.00),
    ('Савицкая', 'Марина', 'Олеговна', date '1994-11-05', 'F', 'МС', '2718281',
    'Московское РУВД г. Минска', date '2020-01-17', '4051194B014PB1', 'г. Борисов', 1,
    'пр. Дзержинского, д. 91, кв. 205', null, '+375 (33) 271-82-81', 'savitskaya@example.by', 8,
    2, 1, 1, false, 3100.50),
    ('Ковальчук', 'Игорь', 'Петрович', date '1958-07-23', 'М', 'АВ', '1618033',
    'Ленинский РОВД г. Бреста', date '2014-08-11', '3230758C027PB8', 'д. Чернавчицы Брестского р-на', 2,
    'ул. Московская, д. 273, кв. 41', '205-17-33', null, null, 2,
    2, 1, 4, true, 1180.00),
    ('Абрамович', 'Елена', 'Сергеевна', date '1987-02-28', 'F', 'КН', '1414213',
    'Октябрьский РОВД г. Гродно', date '2017-06-30', '4280287K003PB2', 'г. Гродно', 5,
    'ул. Ожешко, д. 15, кв. 7', null, '+375 (44) 141-42-13', 'abramovich@example.by', 5,
    3, 1, 1, false, null),
    ('Жуков', 'Дмитрий', 'Александрович', date '1979-09-09', 'М', 'ВМ', '1732050',
    'Железнодорожный РОВД г. Гомеля', date '2016-03-21', '3090979E045PB6', 'г. Жлобин', 4,
    'ул. Советская, д. 97, кв. 118', '255-40-12', '+375 (25) 173-20-50', null, 1,
    2, 2, 1, false, 5600.00),
    ('Шевчук', 'Ольга', 'Николаевна', date '1951-12-01', 'F', 'ВМ', '2236067',
    'Первомайский РОВД г. Витебска', date '2012-10-04', '4011251H019PB3', 'г. Полоцк', 3,
    'пр. Фрунзе, д. 34, кв. 56', '236-70-98', null, null, 3,
    4, 1, 3, true, 920.40);

```

Листинг А.9 – client-service/src/main/java/by/bsuir/bank/client/domain/Citizenship.java

```
package by.bsuir.bank.client.domain;
```

```
import jakarta.persistence.Entity;

/** Справочник: Гражданство. */
@Entity
public class Citizenship extends DictionaryItem {
}
```

Листинг А.10 – client-service/src/main/java/by/bsuir/bank/client/domain/City.java

```
package by.bsuir.bank.client.domain;

import jakarta.persistence.Entity;

/** Справочник: Города (проживания и прописки). */
@Entity
public class City extends DictionaryItem {
}
```

Листинг А.11 – client-service/src/main/java/by/bsuir/bank/client/domain/Client.java

```
package by.bsuir.bank.client.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.LocalDate;

/** Клиент банка – физическое лицо. */
@Entity
@Getter
@Setter
public class Client {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String lastName;
    private String firstName;
    private String middleName;
    private LocalDate birthDate;

    @Enumerated(EnumType.STRING)
    private Sex sex;

    private String passportSeries;
    private String passportNumber;
    private String issuedBy;
    private LocalDate issueDate;
    private String identificationNumber;
    private String birthPlace;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    private City residenceCity;

    private String residenceAddress;
    private String homePhone;
    private String mobilePhone;
    private String email;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    private City registrationCity;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    private MaritalStatus maritalStatus;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
```

```

private Citizenship citizenship;

@ManyToOne(fetch = FetchType.LAZY, optional = false)
private Disability disability;

private boolean pensioner;
private BigDecimal monthlyIncome;

public String getFullName() {
    return lastName + " " + firstName + " " + middleName;
}
}

```

Листинг А.12 – client-service/src/main/java/by/bsuir/bank/client/domain/DictionaryItem.java

```

package by.bsuir.bank.client.domain;

import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.MappedSuperclass;
import lombok.Getter;
import lombok.Setter;

/** Общая структура справочников: идентификатор и наименование. */
@MappedSuperclass
@Getter
@Setter
public abstract class DictionaryItem {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
}

```

Листинг А.13 – client-service/src/main/java/by/bsuir/bank/client/domain/Disability.java

```

package by.bsuir.bank.client.domain;

import jakarta.persistence.Entity;

/** Справочник: Группы инвалидности. */
@Entity
public class Disability extends DictionaryItem {
}

```

Листинг А.14 – client-service/src/main/java/by/bsuir/bank/client/domain/MaritalStatus.java

```

package by.bsuir.bank.client.domain;

import jakarta.persistence.Entity;

/** Справочник: Семейное положение. */
@Entity
public class MaritalStatus extends DictionaryItem {
}

```

Листинг А.15 – client-service/src/main/java/by/bsuir/bank/client/domain/Sex.java

```

package by.bsuir.bank.client.domain;

public enum Sex {
    M, F
}

```

Листинг А.16 – client-service/src/main/java/by/bsuir/bank/client/repository/CitizenshipRepository.java

```
package by.bsuir.bank.client.repository;

import by.bsuir.bank.client.domain.Citizenship;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CitizenshipRepository extends JpaRepository<Citizenship, Long> {
}
```

Листинг А.17 – client-service/src/main/java/by/bsuir/bank/client/repository/CityRepository.java

```
package by.bsuir.bank.client.repository;

import by.bsuir.bank.client.domain.City;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CityRepository extends JpaRepository<City, Long> {
}
```

Листинг А.18 – client-service/src/main/java/by/bsuir/bank/client/repository/ClientRepository.java

```
package by.bsuir.bank.client.repository;

import by.bsuir.bank.client.domain.Client;
import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
import java.util.Optional;

public interface ClientRepository extends JpaRepository<Client, Long> {

    /** Список клиентов, отсортированный по фамилии (затем по имени и отчеству). */
    @EntityGraph(attributePaths = {"residenceCity", "registrationCity", "maritalStatus", "citizenship", "disability"})
    List<Client> findAllByOrderByLastNameAscFirstNameAscMiddleNameAsc();

    @EntityGraph(attributePaths = {"residenceCity", "registrationCity", "maritalStatus", "citizenship", "disability"})
    Optional<Client> findWithDictionariesById(Long id);

    Optional<Client> findByPassportSeriesAndPassportNumber(String series, String number);

    Optional<Client> findByIdentificationNumber(String identificationNumber);
}
```

Листинг А.19 – client-service/src/main/java/by/bsuir/bank/client/repository/DisabilityRepository.java

```
package by.bsuir.bank.client.repository;

import by.bsuir.bank.client.domain.Disability;
import org.springframework.data.jpa.repository.JpaRepository;

public interface DisabilityRepository extends JpaRepository<Disability, Long> {
}
```

Листинг А.20 – client-service/src/main/java/by/bsuir/bank/client/repository/MaritalStatusRepository.java

```
package by.bsuir.bank.client.repository;

import by.bsuir.bank.client.domain.MaritalStatus;
import org.springframework.data.jpa.repository.JpaRepository;

public interface MaritalStatusRepository extends JpaRepository<MaritalStatus, Long> {
}
```

Листинг А.21 – client-service/src/main/java/by/bsuir/bank/client/dto/ApiError.java

```
package by.bsuir.bank.client.dto;

import java.util.Map;

/** Единый формат ошибки: общее сообщение и сообщения по полям формы. */
public record ApiError(String message, Map<String, String> fields) {
}
```

Листинг А.22 – client-service/src/main/java/by/bsuir/bank/client/dto/ClientRequest.java

```
package by.bsuir.bank.client.dto;

import by.bsuir.bank.client.validation.DateText;
import by.bsuir.bank.client.validation.Masks;
import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.Digits;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

import java.math.BigDecimal;

/**
 * Данные формы клиента. Серверная валидация (вторая линия после проверки в браузере):
 * обязательность – @NotBlank/@NotNull, маски – @Pattern, даты – @DateText.
 */
public record ClientRequest(

    @NotBlank(message = REQUIRED) @Size(max = 60, message = TOO_LONG)
    @Pattern(regexp = Masks.NAME, message = LETTERS_ONLY)
    String lastName,

    @NotBlank(message = REQUIRED) @Size(max = 60, message = TOO_LONG)
    @Pattern(regexp = Masks.NAME, message = LETTERS_ONLY)
    String firstName,

    @NotBlank(message = REQUIRED) @Size(max = 60, message = TOO_LONG)
    @Pattern(regexp = Masks.NAME, message = LETTERS_ONLY)
    String middleName,

    @NotBlank(message = REQUIRED) @DateText
    String birthDate,

    @NotBlank(message = REQUIRED) @Pattern(regexp = "^[MF]$", message = "Выберите пол")
    String sex,

    @NotBlank(message = REQUIRED)
    @Pattern(regexp = Masks.PASSPORT_SERIES, message = "Серия – две заглавные латинские буквы")
    String passportSeries,

    @NotBlank(message = REQUIRED)
    @Pattern(regexp = Masks.PASSPORT_NUMBER, message = "Номер паспорта – семь цифр")
    String passportNumber,

    @NotBlank(message = REQUIRED) @Size(max = 200, message = TOO_LONG)
    String issuedBy,

    @NotBlank(message = REQUIRED) @DateText
    String issueDate,

    @NotBlank(message = REQUIRED)
    @Pattern(regexp = Masks.IDENTIFICATION_NUMBER, message = "Формат: 7 цифр, буква, 3 цифры, 2 буквы, цифра")
    String identificationNumber,

    @NotBlank(message = REQUIRED) @Size(max = 200, message = TOO_LONG)
    String birthPlace,

    @NotNull(message = REQUIRED)
    Long residenceCityId,

    @NotBlank(message = REQUIRED) @Size(max = 200, message = TOO_LONG)
    String residenceAddress,

    @Pattern(regexp = Masks.HOME_PHONE, message = "Формат: 293-88-44")
    String homePhone,
```

```

        @Pattern(regexp = Masks.MOBILE_PHONE, message = "Формат: +375 (29) 314-15-92")
        String mobilePhone,

        @Size(max = 100, message = TOO_LONG) @Pattern(regexp = Masks.EMAIL, message = "Некорректный e-mail")
        String email,

        @NotNull(message = REQUIRED)
        Long registrationCityId,

        @NotNull(message = REQUIRED)
        Long maritalStatusId,

        @NotNull(message = REQUIRED)
        Long citizenshipId,

        @NotNull(message = REQUIRED)
        Long disabilityId,

        @NotNull(message = REQUIRED)
        Boolean pensioner,

        @DecimalMin(value = "0.00", message = "Доход не может быть отрицательным")
        @Digits(integer = 12, fraction = 2, message = "Денежная сумма: до 12 цифр и не более 2 знаков после запятой")
        BigDecimal monthlyIncome
    ) {

    public static final String REQUIRED = "Обязательное поле";
    public static final String TOO_LONG = "Слишком длинное значение";
    public static final String LETTERS_ONLY = "Допустимы только буквы";

    /** Пробелы по краям отбрасываются до проверок: «пробел вместо имени» превращается в пустую строку. */
    public ClientRequest {
        lastName = trim(lastName);
        firstName = trim(firstName);
        middleName = trim(middleName);
        birthDate = trim(birthDate);
        passportSeries = trim(passportSeries);
        passportNumber = trim(passportNumber);
        issuedBy = trim(issuedBy);
        issueDate = trim(issueDate);
        identificationNumber = trim(identificationNumber);
        birthPlace = trim(birthPlace);
        residenceAddress = trim(residenceAddress);
        homePhone = trim(homePhone);
        mobilePhone = trim(mobilePhone);
        email = trim(email);
    }

    private static String trim(String value) {
        return value == null ? null : value.trim();
    }
}

```

Листинг А.23 – client-service/src/main/java/by/bsuir/bank/client/dto/ClientResponse.java

```

package by.bsuir.bank.client.dto;

import by.bsuir.bank.client.domain.Client;
import by.bsuir.bank.client.validation.Dates;

import java.math.BigDecimal;

public record ClientResponse(
    Long id,
    String fullName,
    String lastName,
    String firstName,
    String middleName,
    String birthDate,
    String sex,
    String passportSeries,
    String passportNumber,
    String issuedBy,
    String issueDate,
    String identificationNumber,
    String birthPlace,
    Long residenceCityId,
    String residenceCity,

```

```

        String residenceAddress,
        String homePhone,
        String mobilePhone,
        String email,
        Long registrationCityId,
        String registrationCity,
        Long maritalStatusId,
        String maritalStatus,
        Long citizenshipId,
        String citizenship,
        Long disabilityId,
        String disability,
        boolean pensioner,
        BigDecimal monthlyIncome
    ) {

        public static ClientResponse from(Client c) {
            return new ClientResponse(
                c.getId(), c.getFullName(), c.getLastName(), c.getFirstName(), c.getMiddleName(),
                Dates.format(c.getBirthDate()), c.getSex().name(),
                c.getPassportSeries(), c.getPassportNumber(), c.getIssuedBy(), Dates.format(c.getIssueDate()),
                c.getIdentificationNumber(), c.getBirthPlace(),
                c.getResidenceCity().getId(), c.getResidenceCity().getName(), c.getResidenceAddress(),
                c.getHomePhone(), c.getMobilePhone(), c.getEmail(),
                c.getRegistrationCity().getId(), c.getRegistrationCity().getName(),
                c.getMaritalStatus().getId(), c.getMaritalStatus().getName(),
                c.getCitizenship().getId(), c.getCitizenship().getName(),
                c.getDisability().getId(), c.getDisability().getName(),
                c.isPensioner(), c.getMonthlyIncome());
        }
    }
}

```

Листинг А.24 – client-service/src/main/java/by/bsuir/bank/client/dto/Dictionaries.java

```

package by.bsuir.bank.client.dto;

import by.bsuir.bank.client.domain.Citizenship;
import by.bsuir.bank.client.domain.City;
import by.bsuir.bank.client.domain.Disability;
import by.bsuir.bank.client.domain.MaritalStatus;

import java.util.List;

/** Содержимое всех справочников – для выпадающих списков формы. */
public record Dictionaries(
    List<City> cities,
    List<MaritalStatus> maritalStatuses,
    List<Citizenship> citizenships,
    List<Disability> disabilities
) {
}

```

Листинг А.25 – client-service/src/main/java/by/bsuir/bank/client/validation/DateText.java

```

package by.bsuir.bank.client.validation;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;

import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

/** Строка содержит существующую календарную дату ДД.ММ.ГГГГ в диапазоне от 01.01.1900 до сегодняшнего дня. */
@Documented
@Constraint(validatedBy = DateTextValidator.class)
@Target({ElementType.FIELD, ElementType.RECORD_COMPONENT})
@Retention(RetentionPolicy.RUNTIME)
public @interface DateText {

    String message() default "Укажите существующую дату ДД.ММ.ГГГГ не позднее сегодняшней";

    Class<?>[] groups() default {};
}

```

```

Class<? extends Payload>[] payload() default {};
}

```

Листинг А.26 – client-service/src/main/java/by/bsuir/bank/client/validation/DateTextValidator.java

```

package by.bsuir.bank.client.validation;

import jakarta.validation.ConstraintValidator;
import jakarta.validation.ConstraintValidatorContext;

import java.time.LocalDate;

public class DateTextValidator implements ConstraintValidator<DateText, String> {

    private static final LocalDate MIN = LocalDate.of(1900, 1, 1);

    @Override
    public boolean isValid(String value, ConstraintValidatorContext context) {
        if (value == null || value.isBlank()) {
            return true; // обязательность проверяет @NotBlank
        }
        return Dates.parse(value)
            .filter(date -> !date.isBefore(MIN) && !date.isAfter(LocalDate.now()))
            .isPresent();
    }
}

```

Листинг А.27 – client-service/src/main/java/by/bsuir/bank/client/validation/Dates.java

```

package by.bsuir.bank.client.validation;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.time.format.ResolverStyle;
import java.util.Optional;

/** Даты передаются между клиентом и сервисом текстом в формате ДД.ММ.ГГГГ. */
public final class Dates {

    /** STRICT отвергает 31.02.2015 и 29.02.2015 – без него парсер молча «исправил» бы дату на 28.02. */
    public static final DateTimeFormatter FORMAT =
        DateTimeFormatter.ofPattern("dd.MM.uuuu").withResolverStyle(ResolverStyle.STRICT);

    public static Optional<LocalDate> parse(String text) {
        if (text == null) {
            return Optional.empty();
        }
        try {
            return Optional.of(LocalDate.parse(text.trim(), FORMAT));
        } catch (DateTimeParseException e) {
            return Optional.empty();
        }
    }

    public static String format(LocalDate date) {
        return date == null ? null : FORMAT.format(date);
    }

    private Dates() {}
}

```

Листинг А.28 – client-service/src/main/java/by/bsuir/bank/client/validation/Masks.java

```

package by.bsuir.bank.client.validation;

/** Маски полей клиента. Те же выражения продублированы в web-клиенте (static/app.js). */
public final class Masks {

    /** ФИО: только буквы, составные части через дефис, пробел или апостроф. */
    public static final String NAME = "[А-Яа-яЁёА-Зз-з]{?:[-' ][А-Яа-яЁёА-Зз-з]+)*$";
}

```

```

/** Серия паспорта: две заглавные латинские буквы (MP, MC, AB, BM, KH, HB, KB, PP...). */
public static final String PASSPORT_SERIES = "[A-Z]{2}$";

/** Номер паспорта: семь цифр. */
public static final String PASSPORT_NUMBER = "\\d{7}$";

/** Идентификационный номер: 7 цифр, буква, 3 цифры, 2 буквы, цифра — 3140301A001PB5. */
public static final String IDENTIFICATION_NUMBER = "\\d{7}[A-Z]\\d{3}[A-Z]{2}\\d$";

/** Домашний телефон: 293-88-44. Пустая строка допустима — поле необязательное. */
public static final String HOME_PHONE = "^$|^\\d{3}-\\d{2}-\\d{2}$";

/** Мобильный телефон: +375 (29) 314-15-92, коды операторов 25, 29, 33, 44. */
public static final String MOBILE_PHONE = "^$|^\\+375 \\((25|29|33|44)\\) \\d{3}-\\d{2}-\\d{2}$";

public static final String EMAIL = "^$|^([A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,})$";

private Masks() {
}
}

```

Листинг А.29 —

client-service/src/main/java/by/bsuir/bank/client/ClientServiceApplication.java

```

package by.bsuir.bank.client;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ClientServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(ClientServiceApplication.class, args);
    }

}

```

Листинг А.30 —

client-service/src/main/java/by/bsuir/bank/client/service/ClientService.java

```

package by.bsuir.bank.client.service;

import by.bsuir.bank.client.domain.Client;
import by.bsuir.bank.client.domain.Sex;
import by.bsuir.bank.client.dto.ClientRequest;
import by.bsuir.bank.client.dto.ClientResponse;
import by.bsuir.bank.client.repository.CitizenshipRepository;
import by.bsuir.bank.client.repository.CityRepository;
import by.bsuir.bank.client.repository.ClientRepository;
import by.bsuir.bank.client.repository.DisabilityRepository;
import by.bsuir.bank.client.repository.MaritalStatusRepository;
import by.bsuir.bank.client.validation.Dates;
import lombok.RequiredArgsConstructor;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

@Service
@RequiredArgsConstructor
public class ClientService {

    static final String DUPLICATE_PASSPORT = "Клиент с таким паспортом уже зарегистрирован";
    static final String DUPLICATE_IDENTIFICATION = "Клиент с таким идентификационным номером уже зарегистрирован";

    private final ClientRepository clients;
    private final CityRepository cities;
    private final MaritalStatusRepository maritalStatuses;
    private final CitizenshipRepository citizenships;
    private final DisabilityRepository disabilities;

    @Transactional(readOnly = true)

```

```

public List<ClientResponse> findAll() {
    return clients.findAllByOrderByLastNameAscFirstNameAscMiddleNameAsc().stream()
        .map(ClientResponse::from)
        .toList();
}

@Transactional(readOnly = true)
public ClientResponse get(Long id) {
    return clients.findWithDictionariesById(id)
        .map(ClientResponse::from)
        .orElseThrow(() -> notFound(id));
}

@Transactional
public ClientResponse create(ClientRequest request) {
    return save(new Client(), request);
}

@Transactional
public ClientResponse update(Long id, ClientRequest request) {
    return save(clients.findById(id).orElseThrow(() -> notFound(id)), request);
}

@Transactional
public void delete(Long id) {
    if (!clients.existsById(id)) {
        throw notFound(id);
    }
    clients.deleteById(id);
    clients.flush();
}

private ClientResponse save(Client client, ClientRequest request) {
    LocalDate birthDate = Dates.parse(request.birthDate()).orElseThrow();
    LocalDate issueDate = Dates.parse(request.issueDate()).orElseThrow();
    if (issueDate.isBefore(birthDate)) {
        throw new FieldErrorException(HttpStatus.BAD_REQUEST, "Проверьте правильность заполнения полей",
            Map.of("issueDate", "Паспорт не может быть выдан раньше даты рождения"));
    }
    checkUnique(request, client.getId());

    client.setLastName(request.lastName());
    client.setFirstName(request.firstName());
    client.setMiddleName(request.middleName());
    client.setBirthDate(birthDate);
    client.setSex(Sex.valueOf(request.sex()));
    client.setPassportSeries(request.passportSeries());
    client.setPassportNumber(request.passportNumber());
    client.setIssuedBy(request.issuedBy());
    client.setIssueDate(issueDate);
    client.setIdentificationNumber(request.identificationNumber());
    client.setBirthPlace(request.birthPlace());
    client.setResidenceCity(reference(cities, request.residenceCityId(), "residenceCityId"));
    client.setResidenceAddress(request.residenceAddress());
    client.setHomePhone(blankOrNull(request.homePhone()));
    client.setMobilePhone(blankOrNull(request.mobilePhone()));
    client.setEmail(blankOrNull(request.email()));
    client.setRegistrationCity(reference(cities, request.registrationCityId(), "registrationCityId"));
    client.setMaritalStatus(reference(maritalStatuses, request.maritalStatusId(), "maritalStatusId"));
    client.setCitizenship(reference(citizenships, request.citizenshipId(), "citizenshipId"));
    client.setDisability(reference(disabilities, request.disabilityId(), "disabilityId"));
    client.setPensioner(request.pensioner());
    client.setMonthlyIncome(request.monthlyIncome());

    return ClientResponse.from(clients.saveAndFlush(client));
}

/** Паспорт и идентификационный номер уникальны; при редактировании собственная запись клиента не в счёт. */
private void checkUnique(ClientRequest request, Long selfId) {
    Map<String, String> errors = new LinkedHashMap<>();
    clients.findByPassportSeriesAndPassportNumber(request.passportSeries(), request.passportNumber())
        .filter(other -> !other.getId().equals(selfId))
        .ifPresent(other -> {
            errors.put("passportSeries", DUPLICATE_PASSPORT);
            errors.put("passportNumber", DUPLICATE_PASSPORT);
        });
    clients.findByIdentificationNumber(request.identificationNumber())
        .filter(other -> !other.getId().equals(selfId))
        .ifPresent(other -> errors.put("identificationNumber", DUPLICATE_IDENTIFICATION));
    if (!errors.isEmpty()) {
        throw new FieldErrorException(HttpStatus.CONFLICT, "Такой клиент уже есть в базе", errors);
    }
}

```

```

    }

    private static <T> T reference(JpaRepository<T, Long> dictionary, Long id, String field) {
        return dictionary.findById(id).orElseThrow(() -> new FieldErrorException(HttpStatus.BAD_REQUEST,
            "Проверьте правильность заполнения полей", Map.of(field, "Значение отсутствует в справочнике")));
    }

    private static String blankToNull(String value) {
        return value == null || value.isBlank() ? null : value;
    }

    private static NotFoundException notFound(Long id) {
        return new NotFoundException("Клиент с идентификатором " + id + " не найден");
    }
}

```

Листинг A.31 – client- service/src/main/java/by/bsuir/bank/client/service/FieldErrorException.java

```

package by.bsuir.bank.client.service;

import lombok.Getter;
import org.springframework.http.HttpStatus;

import java.util.Map;

/** Ошибка бизнес-правила, привязанная к полям формы (дубликат паспорта, отсутствие записи справочника...). */
@Getter
public class FieldErrorException extends RuntimeException {

    private final HttpStatus status;
    private final Map<String, String> fields;

    public FieldErrorException(HttpStatus status, String message, Map<String, String> fields) {
        super(message);
        this.status = status;
        this.fields = fields;
    }
}

```

Листинг A.32 – client- service/src/main/java/by/bsuir/bank/client/service/NotFoundException.java

```

package by.bsuir.bank.client.service;

public class NotFoundException extends RuntimeException {

    public NotFoundException(String message) {
        super(message);
    }
}

```

Листинг A.33 – client-service/src/main/java/by/bsuir/bank/client/web/ApiExceptionHandler.java

```

package by.bsuir.bank.client.web;

import by.bsuir.bank.client.dto.ApiError;
import by.bsuir.bank.client.dto.ClientRequest;
import by.bsuir.bank.client.service.FieldErrorException;
import by.bsuir.bank.client.service.NotFoundException;
import com.fasterxml.jackson.databind.JsonMappingException;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.LinkedHashMap;
import java.util.Map;

```

```

@RestControllerAdvice
public class ApiExceptionHandler {

    private static final String CHECK_FIELDS = "Проверьте правильность заполнения полей";

    /** Нарушения Bean Validation: по одному сообщению на поле, «Обязательное поле» в приоритете. */
    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ApiError invalid(MethodArgumentNotValidException e) {
        Map<String, String> fields = new LinkedHashMap<>();
        for (FieldError error : e.getBindingResult().getFieldErrors()) {
            String message = error.getDefaultMessage();
            fields.merge(error.getField(), message,
                (old, fresh) -> ClientRequest.REQUIRED.equals(fresh) ? fresh : old);
        }
        return new ApiError(CHECK_FIELDS, fields);
    }

    /** JSON не разобран: например, в денежное поле или в идентификатор справочника пришёл текст. */
    @ExceptionHandler(HttpMessageNotReadableException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ApiError unreadable(HttpMessageNotReadableException e) {
        Map<String, String> fields = new LinkedHashMap<>();
        if (e.getCause() instanceof JsonMappingException mapping && !mapping.getPath().isEmpty()) {
            fields.put(mapping.getPath().get(0).getFieldName(), "Некорректное значение");
        }
        return new ApiError(fields.isEmpty() ? "Некорректный формат запроса" : CHECK_FIELDS, fields);
    }

    @ExceptionHandler(FieldErrorException.class)
    public ResponseEntity<ApiError> business(FieldErrorException e) {
        return ResponseEntity.status(e.getStatus()).body(new ApiError(e.getMessage(), e.getFields()));
    }

    @ExceptionHandler(NotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ApiError notFound(NotFoundException e) {
        return new ApiError(e.getMessage(), Map.of());
    }

    /** Сработало ограничение уникальности в самой базе (гонка двух одновременных запросов). */
    @ExceptionHandler(DataIntegrityViolationException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public ApiError integrity(DataIntegrityViolationException e) {
        return new ApiError("Клиент с таким паспортом или идентификационным номером уже есть в базе", Map.of());
    }
}

```

Листинг А.34 – client-service/src/main/java/by/bsuir/bank/client/web/ClientController.java

```

package by.bsuir.bank.client.web;

import by.bsuir.bank.client.dto.ClientRequest;
import by.bsuir.bank.client.dto.ClientResponse;
import by.bsuir.bank.client.service.ClientService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api/clients")
@RequiredArgsConstructor
public class ClientController {

    private final ClientService service;

    @GetMapping
    public List<ClientResponse> list() {

```

```

        return service.findAll();
    }

    @GetMapping("/{id}")
    public ClientResponse get(@PathVariable Long id) {
        return service.get(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public ClientResponse create(@Valid @RequestBody ClientRequest request) {
        return service.create(request);
    }

    @PutMapping("/{id}")
    public ClientResponse update(@PathVariable Long id, @Valid @RequestBody ClientRequest request) {
        return service.update(id, request);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        service.delete(id);
    }
}

```

Листинг А.35 – client-service/src/main/java/by/bsuir/bank/client/web/DictionaryController.java

```

package by.bsuir.bank.client.web;

import by.bsuir.bank.client.dto.Dictionaries;
import by.bsuir.bank.client.repository.CitizenshipRepository;
import by.bsuir.bank.client.repository.CityRepository;
import by.bsuir.bank.client.repository.DisabilityRepository;
import by.bsuir.bank.client.repository.MaritalStatusRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequiredArgsConstructor
public class DictionaryController {

    private final CityRepository cities;
    private final MaritalStatusRepository maritalStatuses;
    private final CitizenshipRepository citizenships;
    private final DisabilityRepository disabilities;

    @GetMapping("/api/dictionaries")
    public Dictionaries dictionaries() {
        Sort byId = Sort.by("id");
        return new Dictionaries(cities.findAll(Sort.by("name")), maritalStatuses.findAll(byId),
            citizenships.findAll(byId), disabilities.findAll(byId));
    }
}

```

Листинг А.36 – client-service/src/main/resources/static/index.html

```

<!doctype html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Клиенты – Банк «Решение»</title>
    <link rel="stylesheet" href="/ui/bank.css">
</head>
<body>
    <div id="app" v-cloak></div>

    <script src="/webjars/vue/dist/vue.global.prod.js"></script>
    <script src="/ui/bank.js"></script>
    <script src="/app.js"></script>
</body>
</html>

```

Листинг А.37 – client-service/src/main/resources/static/app.js

```
/* Web-клиент модуля «Работа с клиентами»: форма «Список клиентов» и форма добавления/редактирования. */

const REQUIRED = 'Обязательное поле';
const NAME = /^[А-Яа-яЁёА-Za-z](?:[-' ][А-Яа-яЁёА-Za-z]+)*$/;
const letters = v => NAME.test(v) || 'Допустимы только буквы';

function pastDate(v) {
  const date = Bank.parseDate(v);
  if (!date) return 'Такой даты не существует, формат ДД.ММ.ГГГГ';
  if (date > new Date()) return 'Дата не может быть в будущем';
  if (date.getFullYear() < 1900) return 'Дата не может быть раньше 1900 года';
  return true;
}

/*
 * Описание полей формы – единственное место, где заданы обязательность, маска ввода и проверка значения.
 * Первичная (клиентская) валидация: required – обязательность, mask – маска ввода, check – проверка формата.
 * Те же правила повторно проверяет сервис (ClientRequest + Masks).
 */
const SECTIONS = [
  { title: 'Личные данные', fields: [
    { name: 'lastName', label: 'Фамилия', required: true, max: 60, check: letters },
    { name: 'firstName', label: 'Имя', required: true, max: 60, check: letters },
    { name: 'middleName', label: 'Отчество', required: true, max: 60, check: letters },
    { name: 'birthDate', label: 'Дата рождения', required: true, mask: '##.##.####', placeholder: 'ДД.ММ.ГГГГ', check: pastDate },
    { name: 'sex', label: 'Пол', required: true, type: 'radio', options: [{ id: 'M', name: 'Мужской' }, { id: 'F', name: 'Женский' }] },
    { name: 'birthPlace', label: 'Место рождения', required: true, max: 200 },
  ]},
  { title: 'Паспортные данные', fields: [
    { name: 'passportSeries', label: 'Серия паспорта', required: true, mask: 'AA', placeholder: 'MP',
      check: v => /^[A-Z]{2}$/.test(v) || 'Серия – две заглавные латинские буквы' },
    { name: 'passportNumber', label: '№ паспорта', required: true, mask: '#####', placeholder: '1234567',
      check: v => /\d{7}$/.test(v) || 'Номер паспорта – семь цифр' },
    { name: 'identificationNumber', label: 'Идентификационный номер', required: true, mask: '#####A####AA#', placeholder: '3140301A001PB5',
      check: v => /\d{7}[A-Z]\d{3}[A-Z]{2}\d$/.test(v) || 'Формат: 7 цифр, буква, 3 цифры, 2 буквы, цифра' },
    { name: 'issuedBy', label: 'Кем выдан', required: true, max: 200, wide: true },
    { name: 'issueDate', label: 'Дата выдачи', required: true, mask: '##.##.####', placeholder: 'ДД.ММ.ГГГГ',
      check: (v, form) => {
        const valid = pastDate(v);
        if (valid !== true) return valid;
        const birth = Bank.parseDate(form.birthDate);
        return !birth || Bank.parseDate(v) >= birth || 'Паспорт не может быть выдан раньше даты рождения';
      } },
  ]},
  { title: 'Проживание и контакты', fields: [
    { name: 'residenceCityId', label: 'Город фактического проживания', required: true, type: 'select', dictionary: 'cities' },
    { name: 'residenceAddress', label: 'Адрес фактического проживания', required: true, max: 200, wide: true },
    { name: 'registrationCityId', label: 'Город прописки', required: true, type: 'select', dictionary: 'cities' },
    { name: 'homePhone', label: 'Телефон домашний', mask: '###-##-##', placeholder: '293-88-44',
      check: v => /\d{3}-\d{2}-\d{2}$/.test(v) || 'Формат: 293-88-44' },
    { name: 'mobilePhone', label: 'Телефон мобильный', mask: '+375 (##) ###-##-##', placeholder: '+375 (29) 314-15-92',
      check: v => /\^+375 \((25|29|33|44)\) \d{3}-\d{2}-\d{2}$/.test(v) || 'Формат: +375 (29) 314-15-92, код 25, 29, 33 или 44' },
    { name: 'email', label: 'E-mail', max: 100, placeholder: 'name@example.by',
      check: v => /^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/.test(v) || 'Некорректный e-mail' },
  ]},
  { title: 'Социальный статус', fields: [
    { name: 'maritalStatusId', label: 'Семейное положение', required: true, type: 'select', dictionary: 'maritalStatuses' },
    { name: 'citizenshipId', label: 'Гражданство', required: true, type: 'select', dictionary: 'citizenships' },
    { name: 'disabilityId', label: 'Инвалидность', required: true, type: 'select', dictionary: 'disabilities' },
    { name: 'monthlyIncome', label: 'Ежемесячный доход, BYN', placeholder: '0.00',
      check: v => /\d{1,12}([.]\d{1,2})?$/ .test(v) || 'Денежная сумма: только цифры и не более 2 знаков после запятой' },
    { name: 'pensioner', label: 'Пенсионер', type: 'checkbox' },
  ]},
];

const FIELDS = SECTIONS.flatMap(section => section.fields);

/* Проверка формы перед отправкой: возвращает { имяПоля: сообщение }. Пустой объект – ошибок нет. */
function validate(form) {
  const errors = {};
  for (const field of FIELDS) {
    const raw = form[field.name];
    const value = typeof raw === 'string' ? raw.trim() : raw;
    const empty = value === null || value === undefined || value === '';
    if (empty) {
      if (field.required) errors[field.name] = REQUIRED;
      continue;
    }
  }
}
```

```

    const result = field.check ? field.check(value, form) : true;
    if (result !== true) errors[field.name] = result;
  }
  return errors;
}

function emptyForm() {
  const form = {};
  for (const field of FIELDS) form[field.name] = field.type === 'checkbox' ? false : (field.type === 'select' ? null : '');
  return form;
}

/* Тело REST-запроса: строки без пробелов по краям, пустые необязательные поля — null, доход — число. */
function toPayload(form) {
  const payload = {};
  for (const field of FIELDS) {
    const raw = form[field.name];
    const value = typeof raw === 'string' ? raw.trim() : raw;
    payload[field.name] = value === '' ? null : value;
  }
  if (payload.monthlyIncome !== null) payload.monthlyIncome = Number(String(payload.monthlyIncome).replace(',', '.'));
  return payload;
}

const app = Vue.createApp({
  data() {
    return {
      view: 'list',          // list | form
      clients: [],
      dictionaries: { cities: [], maritalStatuses: [], citizenships: [], disabilities: [] },
      sections: SECTIONS,
      form: emptyForm(),
      editingId: null,
      errors: {},
      formError: '',
      notice: '',
      loadError: '',
      saving: false,
    };
  },
  async mounted() {
    try {
      this.dictionaries = await Bank.api('GET', '/api/dictionaries');
      await this.reload();
    } catch (e) {
      this.loadError = e.message;
    }
  },
  methods: {
    async reload() {
      this.clients = await Bank.api('GET', '/api/clients');
    },
    openCreate() {
      this.form = emptyForm();
      this.editingId = null;
      this.showForm();
    },
    async openEdit(id) {
      const client = await Bank.api('GET', '/api/clients/' + id);
      const form = emptyForm();
      for (const field of FIELDS) form[field.name] = client[field.name] ?? form[field.name];
      if (client.monthlyIncome !== null) form.monthlyIncome = Number(client.monthlyIncome).toFixed(2);
      this.form = form;
      this.editingId = id;
      this.showForm();
    },
    showForm() {
      this.errors = {};
      this.formError = '';
      this.notice = '';
      this.view = 'form';
    },
    async save() {
      this.errors = validate(this.form);
      this.formError = Object.keys(this.errors).length ? 'Форма заполнена с ошибками — исправьте отмеченные поля' : '';
      if (this.formError) return window.scrollTo(0, 0);
      this.saving = true;
      try {
        const payload = toPayload(this.form);
        if (this.editingId === null) await Bank.api('POST', '/api/clients', payload);
        else await Bank.api('PUT', '/api/clients/' + this.editingId, payload);
        await this.reload();
      }
    }
  }
});

```

```

    this.notice = this.editingId === null ? 'Клиент добавлен' : 'Изменения сохранены';
    this.view = 'list';
  } catch (e) {
    this.errors = e.fields; // сообщения серверной валидации показываются у тех же полей
    this.formError = e.message;
    window.scrollTo(0, 0);
  } finally {
    this.saving = false;
  }
},
async remove(client) {
  if (!confirm('Удалить клиента ' + client.fullName + '?')) return;
  try {
    await Bank.api('DELETE', '/api/clients/' + client.id);
    await this.reload();
    this.notice = 'Клиент удалён';
    this.view = 'list';
  } catch (e) {
    this.notice = '';
    this.loadError = e.message;
  }
},
options(field) {
  return field.options || this.dictionaries[field.dictionary];
},
money: Bank.money,
},
template: `
<bank-nav active="clients"></bank-nav>
<main>
  <div v-if="loadError" class="alert error">{{ loadError }}</div>

  <section v-if="view === 'list'">
    <div class="toolbar">
      <div>
        <h1>Список клиентов</h1>
        <p class="subtitle">Всего клиентов: <span id="clients-count">{{ clients.length }}</span>. Сортировка по фамилии.</p>
      </div>
      <button id="btn-add" class="primary" @click="openCreate">Добавить клиента</button>
    </div>
    <div v-if="notice" id="notice" class="alert success">{{ notice }}</div>
    <div class="card">
      <table id="clients">
        <thead>
          <tr>
            <th>ФИО</th><th>Дата рождения</th><th>Паспорт</th><th>Идент. номер</th>
            <th>Город проживания</th><th>Мобильный телефон</th><th class="num">Доход, BYN</th><th></th>
          </tr>
        </thead>
        <tbody>
          <tr v-for="client in clients" :key="client.id">
            <td class="fio">{{ client.fullName }}</td>
            <td class="nowrap">{{ client.birthDate }}</td>
            <td class="mono">{{ client.passportSeries }} {{ client.passportNumber }}</td>
            <td class="mono">{{ client.identificationNumber }}</td>
            <td>{{ client.residenceCity }}</td>
            <td class="nowrap">{{ client.mobilePhone }}</td>
            <td class="num">{{ money(client.monthlyIncome) }}</td>
            <td class="actions">
              <button class="small edit" @click="openEdit(client.id)">Изменить</button>
              <button class="small danger delete" @click="remove(client)">Удалить</button>
            </td>
          </tr>
        </tbody>
      </table>
      <div v-if="!clients.length" class="empty">Клиентов пока нет</div>
    </div>
  </section>

  <section v-else>
    <h1>{{ editingId === null ? 'Новый клиент' : 'Редактирование клиента' }}</h1>
    <p class="subtitle">Поля, отмеченные <span style="color: var(--bad)">*</span>, обязательны для заполнения.</p>
    <div v-if="formError" id="form-alert" class="alert error">{{ formError }}</div>
    <form class="card" novalidate @submit.prevent="save">
      <fieldset v-for="section in sections" :key="section.title">
        <legend>{{ section.title }}</legend>
        <div class="form-grid">
          <div v-for="field in section.fields" :key="field.name"
            class="field" :class="{ invalid: errors[field.name], wide: field.wide }">
            <label :for="'f-' + field.name">{{ field.label }} <span v-if="field.required" class="req">*</span></label>

            <select v-if="field.type === 'select'" :id="'f-' + field.name" v-model="form[field.name]">

```

```

        <option :value="null">-- выберите --</option>
        <option v-for="item in options(field)" :key="item.id" :value="item.id">{{ item.name }}</option>
    </select>

    <div v-else-if="field.type === 'radio'" class="choices">
        <label v-for="item in options(field)" :key="item.id">
            <input type="radio" :id="'f-' + field.name + '-' + item.id" :name="field.name"
                :value="item.id" v-model="form[field.name]"> {{ item.name }}
        </label>
    </div>

    <div v-else-if="field.type === 'checkbox'" class="choices">
        <label><input type="checkbox" :id="'f-' + field.name" v-model="form[field.name]"> да</label>
    </div>

    <input v-else-if="field.mask" type="text" :id="'f-' + field.name" v-model="form[field.name]"
        v-mask="field.mask" :placeholder="field.placeholder" autocomplete="off">

    <input v-else type="text" :id="'f-' + field.name" v-model="form[field.name]"
        :maxlength="field.max" :placeholder="field.placeholder" autocomplete="off">

    <div v-if="errors[field.name]" class="error" :id="'e-' + field.name">{{ errors[field.name] }}</div>
    </div>
</div>
</fieldset>
<div class="form-actions">
    <button id="btn-save" type="submit" class="primary" :disabled="saving">Сохранить</button>
    <button id="btn-cancel" type="button" @click="view = 'list'">Отмена</button>
    <span style="flex: 1"></span>
    <button v-if="editingId !== null" id="btn-delete" type="button" class="danger"
        @click="remove({ id: editingId, fullName: form.lastName + ' ' + form.firstName })">Удалить клиента</button>
</div>
</form>
</section>
</main>',
});

app.component('bank-nav', Bank.navComponent);
app.directive('mask', Bank.maskDirective);
app.mount('#app');

```

Листинг А.38 – client-service/src/test/java/by/bsuir/bank/client/ClientApiIT.java

```

package by.bsuir.bank.client;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.MethodSource;
import org.junit.jupiter.params.provider.ValueSource;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.ResultActions;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.Map;

import static org.assertj.core.api.Assertions.assertThat;
import static org.hamcrest.Matchers.contains;
import static org.hamcrest.Matchers.hasSize;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.put;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/**
 * Интеграционные тесты REST-сервиса на базе H2 в памяти: запрос идёт мимо web-клиента,
 * поэтому проверяется именно серверная валидация. После каждого запроса состояние
 * таблицы client проверяется SQL-запросом «прямо в базе».
 */
// своя база на каждый класс тестов: контексты не делят состояние
@SpringBootTest(properties = "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1")

```

```

@AutoConfigureMockMvc
@Transactional
class ClientApiIT {

    private static final int SEEDDED = 6;

    @Autowired
    private MockMvc mvc;
    @Autowired
    private JdbcTemplate jdbc;

    @Test
    @DisplayName("База заполняется при старте, список отсортирован по фамилии")
    void listIsSortedByLastName() throws Exception {
        mvc.perform(get("/api/clients"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$", hasSize(SEEDDED)))
            .andExpect(jsonPath("$[*].lastName",
                contains("Абрамович", "Жуков", "Ковальчук", "Лебедев", "Савицкая", "Шевчук")));
    }

    @Test
    @DisplayName("Справочники отдаются для выпадающих списков, городов не меньше пяти")
    void dictionariesAreAvailable() throws Exception {
        mvc.perform(get("/api/dictionaries"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.cities.length()").value(8))
            .andExpect(jsonPath("$.maritalStatuses.length()").value(4))
            .andExpect(jsonPath("$.citizenships.length()").value(5))
            .andExpect(jsonPath("$.disabilities.length()").value(4));
    }

    @Test
    @DisplayName("n.1 Повторный ввод тех же данных – ошибка, в базе один клиент")
    void sameClientTwice() throws Exception {
        create(TestClients.valid()).andExpect(status().isCreated());

        create(TestClients.valid())
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.fields.passportNumber").exists())
            .andExpect(jsonPath("$.fields.identificationNumber").exists());

        assertThat(count("last_name = 'Иванов'")).isEqualTo(1);
    }

    @Test
    @DisplayName("n.2 Другой клиент с тем же паспортом – ошибка, паспорт в базе один")
    void anotherClientWithSamePassport() throws Exception {
        create(TestClients.valid()).andExpect(status().isCreated());

        Map<String, Object> other = TestClients.valid();
        other.put("lastName", "Петров");
        other.put("firstName", "Пётр");
        other.put("identificationNumber", "3010185A001PB9");
        create(other)
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.fields.passportSeries").exists())
            .andExpect(jsonPath("$.fields.passportNumber").exists())
            .andExpect(jsonPath("$.fields.identificationNumber").doesNotExist());

        assertThat(count("passport_series = 'MP' and passport_number = '7654321'")).isEqualTo(1);
        assertThat(count("last_name = 'Петров'")).isZero();
    }

    @Test
    @DisplayName("n.3 Другой клиент с тем же идентификационным номером – ошибка")
    void anotherClientWithSameIdentificationNumber() throws Exception {
        create(TestClients.valid()).andExpect(status().isCreated());

        Map<String, Object> other = TestClients.valid();
        other.put("lastName", "Петров");
        other.put("passportNumber", "1111111");
        create(other)
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.fields.identificationNumber").exists())
            .andExpect(jsonPath("$.fields.passportNumber").doesNotExist());

        assertThat(count("identification_number = '3170590A077PB4'")).isEqualTo(1);
    }

    @Test
    @DisplayName("n.4 Фамилия «1234» – ошибка, клиента 1234 в базе нет")

```

```

void digitsInsteadOfLastName() throws Exception {
    create(TestClients.with("lastName", "1234"))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.fields.lastName").value("Допустимы только буквы"));

    assertThat(count("last_name = '1234'")).isZero();
}

@Test
@DisplayName("н.5 Пробел вместо имени – ошибка, клиент не записан")
void spaceInsteadOfFirstName() throws Exception {
    create(TestClients.with("firstName", " "))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.fields.firstName").value("Обязательное поле"));

    assertThat(count("last_name = 'Иванов'")).isZero();
}

@ParameterizedTest(name = "дата рождения «{0}»")
@ValueSource(strings = {"не дата", "31.02.2015", "29.02.2015"})
@DisplayName("н.6 Текст, не являющийся датой, – ошибка")
void birthDateIsNotDate(String birthDate) throws Exception {
    create(TestClients.with("birthDate", birthDate))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.fields.birthDate").exists());

    assertThat(count("last_name = 'Иванов'")).isZero();
}

@ParameterizedTest(name = "{0} = «{1}»")
@CsvSource(delimiter = '|', value = {
    "passportSeries|мп",
    "passportNumber|12345",
    "identificationNumber|3170590-077PB4",
    "homePhone|2014567",
    "mobilePhone|+375 (17) 765-43-21",
    "issueDate|30.02.2018",
    "email|ivanov@"
})
@DisplayName("н.6 Поля с маской проверяются на сервере")
void maskedFieldsAreValidatedByService(String field, String value) throws Exception {
    create(TestClients.with(field, value))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.fields." + field).exists());

    assertThat(count("last_name = 'Иванов'")).isZero();
}

@ParameterizedTest(name = "без поля {0}")
@MethodSource("requiredFields")
@DisplayName("н.7 Без любого обязательного поля запись не выполняется")
void requiredFieldIsMissing(String field) throws Exception {
    Map<String, Object> client = TestClients.valid();
    client.remove(field);

    create(client)
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.fields." + field).value("Обязательное поле"));

    assertThat(count("1 = 1")).isEqualTo(SEEDED);
}

static List<String> requiredFields() {
    return TestClients.REQUIRED_FIELDS;
}

@Test
@DisplayName("н.8 Только обязательные поля – клиент записан")
void onlyRequiredFields() throws Exception {
    create(TestClients.onlyRequired())
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.id").isNumber())
        .andExpect(jsonPath("$.fullName").value("Иванов Иван Иванович"))
        .andExpect(jsonPath("$.mobilePhone").isEmpty())
        .andExpect(jsonPath("$.monthlyIncome").isEmpty());

    assertThat(count("last_name = 'Иванов' and home_phone is null and email is null")).isEqualTo(1);
}

@Test
@DisplayName("Текст в денежном поле – ошибка разбора запроса с указанием поля")
void textInMoneyField() throws Exception {

```

```

        create(TestClients.with("monthlyIncome", "много"))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.fields.monthlyIncome").exists());
    }

    @Test
    @DisplayName("Редактирование: изменения сохраняются, чужой паспорт занять нельзя")
    void update() throws Exception {
        long id = jdbc.queryForObject("select id from client where last_name = 'Жуков'", Long.class);

        Map<String, Object> changed = TestClients.valid();
        changed.put("lastName", "Жуков-Новый");
        mvc.perform(put("/api/clients/" + id).contentType(MediaType.APPLICATION_JSON).content(TestClients.json(changed)))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.lastName").value("Жуков-Новый"));
        assertThat(count("last_name = 'Жуков-Новый' and passport_number = '7654321'").isEqualTo(1);

        // паспорт клиента Лебедевич (MP 3141592) уже занят
        changed.put("passportNumber", "3141592");
        mvc.perform(put("/api/clients/" + id).contentType(MediaType.APPLICATION_JSON).content(TestClients.json(changed)))
            .andExpect(status().isConflict());
    }

    @Test
    @DisplayName("Удаление клиента")
    void remove() throws Exception {
        long id = jdbc.queryForObject("select id from client where last_name = 'Шевчук'", Long.class);

        mvc.perform(delete("/api/clients/" + id)).andExpect(status().isNoContent());
        assertThat(count("1 = 1")).isEqualTo(SEEDED - 1);

        mvc.perform(delete("/api/clients/" + id)).andExpect(status().isNotFound());
        mvc.perform(get("/api/clients/" + id)).andExpect(status().isNotFound());
    }

    private ResultActions create(Map<String, Object> client) throws Exception {
        return mvc.perform(post("/api/clients").contentType(MediaType.APPLICATION_JSON).content(TestClients.json(client)));
    }

    /** Проверка «прямо в базе»: SQL-запрос к таблице client в обход сервиса и репозитория. */
    private int count(String where) {
        return jdbc.queryForObject("select count(*) from client where " + where, Integer.class);
    }
}

```

Листинг А.39 –

client-service/src/test/java/by/bsuir/bank/client/ClientRequestValidationTest.java

```

package by.bsuir.bank.client;

import by.bsuir.bank.client.dto.ClientRequest;
import jakarta.validation.ConstraintViolation;
import jakarta.validation.Validation;
import jakarta.validation.Validator;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.MethodSource;

import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.stream.Collectors;

import static org.assertj.core.api.Assertions.assertThat;

/** Модульные тесты серверной валидации формы: маски и обязательность полей (пп. 4–8 программы защиты). */
class ClientRequestValidationTest {

    private final Validator validator = Validation.buildDefaultValidatorFactory().getValidator();

    @Test
    void validFormHasNoViolations() {
        assertThat(invalidFields(TestClients.valid())).isEmpty();
    }

    @Test
    void optionalFieldsMayBeOmittedOrEmpty() {
        assertThat(invalidFields(TestClients.onlyRequired())).isEmpty();
    }
}

```

```

        Map<String, Object> blank = TestClients.valid();
        blank.put("homePhone", "");
        blank.put("mobilePhone", " ");
        blank.put("email", "");
        blank.put("monthlyIncome", null);
        assertThat(InvalidFields(blank)).isEmpty();
    }

    @ParameterizedTest(name = "без поля {0}")
    @MethodSource("requiredFields")
    void everyRequiredFieldIsChecked(String field) {
        assertThat(InvalidFields(TestClients.with(field, null))).containsExactly(field);
    }

    static List<String> requiredFields() {
        return TestClients.REQUIRED_FIELDS;
    }

    @ParameterizedTest(name = "{0} = «{1}»")
    @CsvSource(delimiter = '|', ignoreLeadingAndTrailingWhitespaces = false, value = {
        "lastName|1234",
        "lastName|Иванов2",
        "lastName|Иванов-",
        "firstName| ",
        "firstName|Иван!",
        "middleName|_",
        "sex|X",
        "passportSeries|mp",
        "passportSeries|M1",
        "passportSeries|MPP",
        "passportNumber|123456",
        "passportNumber|12345678",
        "passportNumber|12A4567",
        "identificationNumber|3170590A077PB",
        "identificationNumber|3170590a077pb4",
        "identificationNumber|31705900077PB4",
        "homePhone|2014567",
        "homePhone|201-45-6",
        "mobilePhone|+375 (17) 765-43-21",
        "mobilePhone|80297654321",
        "mobilePhone|+375 (29) 765-43-2",
        "email|ivanov@",
        "email|ivanov.example.by",
        "birthDate|29.02.2015",
        "issueDate|31.02.2015",
    })
    void maskedFieldsRejectWrongFormat(String field, String value) {
        assertThat(InvalidFields(TestClients.with(field, value))).containsExactly(field);
    }

    @ParameterizedTest
    @CsvSource({"-1", "0.001", "1234567890123"})
    void monthlyIncomeIsMoney(String income) {
        assertThat(InvalidFields(TestClients.with("monthlyIncome", income))).containsExactly("monthlyIncome");
    }

    @Test
    void compositeNamesAreAllowed() {
        Map<String, Object> client = TestClients.valid();
        client.put("lastName", "Петрова-Водкина");
        client.put("firstName", "Анна Мария");
        client.put("middleName", "Ивановна ");
        ClientRequest request = TestClients.request(client);
        assertThat(validator.validate(request)).isEmpty();
        assertThat(request.middleName()).isEqualTo("Ивановна");
    }

    private Set<String> invalidFields(Map<String, Object> fields) {
        return validator.validate(TestClients.request(fields)).stream()
            .map(ConstraintViolation::getPropertyPath)
            .map(Object::toString)
            .collect(Collectors.toSet());
    }
}

```

Листинг А.40 – client-service/src/test/java/by/bsuir/bank/client/ClientServiceTest.java

```
package by.bsuir.bank.client;
```

```

import by.bsuir.bank.client.domain.Citizenship;
import by.bsuir.bank.client.domain.City;
import by.bsuir.bank.client.domain.Client;
import by.bsuir.bank.client.domain.Disability;
import by.bsuir.bank.client.domain.MaritalStatus;
import by.bsuir.bank.client.dto.ClientResponse;
import by.bsuir.bank.client.repository.CitizenshipRepository;
import by.bsuir.bank.client.repository.CityRepository;
import by.bsuir.bank.client.repository.ClientRepository;
import by.bsuir.bank.client.repository.DisabilityRepository;
import by.bsuir.bank.client.repository.MaritalStatusRepository;
import by.bsuir.bank.client.service.ClientService;
import by.bsuir.bank.client.service.FieldErrorException;
import by.bsuir.bank.client.service.NotFoundException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.mockito.junit.jupiter.MockitoSettings;
import org.mockito.quality.Strictness;
import org.springframework.http.HttpStatus;

import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.anyLong;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

/** Модульные тесты бизнес-правил сервиса на заглушках репозитория (пп. 1–3 программы защиты). */
@ExtendWith(MockitoExtension.class)
@MockitoSettings(strictness = Strictness.LENIENT)
class ClientServiceTest {

    @Mock
    private ClientRepository clients;
    @Mock
    private CityRepository cities;
    @Mock
    private MaritalStatusRepository maritalStatuses;
    @Mock
    private CitizenshipRepository citizenships;
    @Mock
    private DisabilityRepository disabilities;

    @InjectMocks
    private ClientService service;

    @BeforeEach
    void dictionaries() {
        when(cities.findById(anyLong())).thenReturn(Optional.of(named(new City(), "Минск")));
        when(maritalStatuses.findById(anyLong())).thenReturn(Optional.of(named(new MaritalStatus(), "Холост")));
        when(citizenships.findById(anyLong())).thenReturn(Optional.of(named(new Citizenship(), "Республика Беларусь")));
        when(disabilities.findById(anyLong())).thenReturn(Optional.of(named(new Disability(), "Нет")));
        when(clients.saveAndFlush(any(Client.class))).thenAnswer(call -> {
            Client client = call.getArgument(0);
            client.setId(100L);
            return client;
        });
    }

    @Test
    void savesNewClient() {
        ClientResponse saved = service.create(TestClients.request(TestClients.valid()));

        assertThat(saved.id()).isEqualTo(100L);
        assertThat(saved.fullName()).isEqualTo("Иванов Иван Иванович");
        assertThat(saved.birthDate()).isEqualTo("17.05.1990");
        assertThat(saved.residenceCity()).isEqualTo("Минск");
    }

    @Test
    void emptyOptionalFieldsAreStoredAsNull() {
        var fields = TestClients.valid();
        fields.put("homePhone", "");
        fields.put("email", " ");
    }

```

```

        ClientResponse saved = service.create(TestClients.request(fields));

        assertThat(saved.homePhone()).isNull();
        assertThat(saved.email()).isNull();
    }

    @Test
    void rejectsSecondClientWithSamePassport() {
        when(clients.findByPassportSeriesAndPassportNumber("MP", "7654321")).thenReturn(Optional.of(existing(7L)));

        assertThatThrownBy(() -> service.create(TestClients.request(TestClients.valid())))
            .isInstanceOfSatisfying(FieldErrorException.class, e -> {
                assertThat(e.getStatus()).isEqualTo(HttpStatus.CONFLICT);
                assertThat(e.getFields()).containsOnlyKeys("passportSeries", "passportNumber");
            });
        verify(clients, never()).saveAndFlush(any());
    }

    @Test
    void rejectsSecondClientWithSameIdentificationNumber() {
        when(clients.findByIdentificationNumber("3170590A077PB4")).thenReturn(Optional.of(existing(7L)));

        assertThatThrownBy(() -> service.create(TestClients.request(TestClients.valid())))
            .isInstanceOfSatisfying(FieldErrorException.class,
                e -> assertThat(e.getFields()).containsOnlyKeys("identificationNumber"));
        verify(clients, never()).saveAndFlush(any());
    }

    @Test
    void clientMayKeepOwnPassportOnUpdate() {
        Client self = existing(7L);
        when(clients.findById(7L)).thenReturn(Optional.of(self));
        when(clients.findByPassportSeriesAndPassportNumber("MP", "7654321")).thenReturn(Optional.of(self));
        when(clients.findByIdentificationNumber("3170590A077PB4")).thenReturn(Optional.of(self));

        service.update(7L, TestClients.request(TestClients.with("lastName", "Петров")));

        assertThat(self.getLastName()).isEqualTo("Петров");
        verify(clients).saveAndFlush(self);
    }

    @Test
    void passportCannotBeIssuedBeforeBirth() {
        var fields = TestClients.with("issueDate", "16.05.1990");

        assertThatThrownBy(() -> service.create(TestClients.request(fields)))
            .isInstanceOfSatisfying(FieldErrorException.class,
                e -> assertThat(e.getFields()).containsOnlyKeys("issueDate"));
    }

    @Test
    void rejectsValueMissingInDictionary() {
        when(cities.findById(99L)).thenReturn(Optional.empty());

        assertThatThrownBy(() -> service.create(TestClients.request(TestClients.with("residenceCityId", 99))))
            .isInstanceOfSatisfying(FieldErrorException.class,
                e -> assertThat(e.getFields()).containsOnlyKeys("residenceCityId"));
    }

    @Test
    void deletingUnknownClientFails() {
        when(clients.existsById(5L)).thenReturn(false);

        assertThatThrownBy(() -> service.delete(5L)).isInstanceOf(NotFoundException.class);
        verify(clients, never()).deleteById(any());
    }

    private static Client existing(Long id) {
        Client client = new Client();
        client.setId(id);
        return client;
    }

    private static <T extends by.bsuir.bank.client.domain.DictionaryItem> T named(T item, String name) {
        item.setId(1L);
        item.setName(name);
        return item;
    }
}

```

Листинг А.41 – client-service/src/test/java/by/bsuir/bank/client/ClientUiIT.java

```
package by.bsuir.bank.client;

import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.Assumptions;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.ValueSource;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.web.server.LocalServerPort;
import org.springframework.jdbc.core.JdbcTemplate;

import java.time.Duration;
import java.util.List;
import java.util.Map;

import static org.assertj.core.api.Assertions.assertThat;

/**
 * Функциональные тесты через WebDriver: настоящий браузер (Chrome без окна) открывает web-клиент
 * поднятого сервиса и проходит программу защиты. Результат каждого шага сверяется с таблицей client в H2.
 */
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT,
    properties = {"spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1"})
class ClientUiIT {

    private static WebDriver driver;

    @LocalServerPort
    private int port;
    @Autowired
    private JdbcTemplate jdbc;

    private WebDriverWait wait;

    @BeforeAll
    static void startBrowser() {
        try {
            ChromeOptions options = new ChromeOptions();
            options.addArguments("--headless=new", "--window-size=1400,1600", "--lang=ru");
            driver = new ChromeDriver(options);
        } catch (Exception e) {
            Assumptions.abort("Chrome недоступен, функциональные тесты пропущены: " + e.getMessage());
        }
    }

    @AfterAll
    static void stopBrowser() {
        if (driver != null) {
            driver.quit();
        }
    }

    @BeforeEach
    void openNewClientForm() {
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        driver.get("http://localhost:" + port + "/");
        wait.until(ExpectedConditions.numberOfElementsToBeMoreThan(By.cssSelector("#clients tbody tr"), 0));
        driver.findElement(By.id("btn-add")).click();
        wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("btn-save")));
    }

    @AfterEach
    void removeCreatedClients() {
        jdbc.update("delete from client where id > 6");
    }
}
```

```

@Test
@DisplayName("Форма «Список клиентов» отсортирована по фамилии")
void listIsSortedByLastName() {
    driver.findElement(By.id("btn-cancel")).click();
    List<String> names = driver.findElements(By.cssSelector("#clients td.fio")).stream()
        .map(WebElement::getText).toList();
    assertThat(names).hasSize(6).isSorted();
    assertThat(names.get(3)).isEqualTo("Лебедев Артем Владимирович");
}

@Test
@DisplayName("п.1 Те же данные ещё раз – сообщение об ошибке, в базе один клиент")
void sameClientTwice() {
    fill(TestClients.valid());
    save();
    waitForNotice("Клиент добавлен");

    driver.findElement(By.id("btn-add")).click();
    fill(TestClients.valid());
    save();

    assertThat(formAlert()).isEqualTo("Такой клиент уже есть в базе");
    assertThat(error("passportNumber")).isEqualTo("Клиент с таким паспортом уже зарегистрирован");
    assertThat(count("last_name = 'Иванов'")).isEqualTo(1);
}

@Test
@DisplayName("п.2 Другой клиент с паспортом студента – сообщение об ошибке")
void anotherClientWithSamePassport() {
    Map<String, Object> other = TestClients.valid();
    other.put("passportSeries", "MP");
    other.put("passportNumber", "3141592"); // паспорт клиента Лебедев из начальных данных
    fill(other);
    save();

    assertThat(error("passportSeries")).isEqualTo("Клиент с таким паспортом уже зарегистрирован");
    assertThat(count("passport_series = 'MP' and passport_number = '3141592'")).isEqualTo(1);
    assertThat(count("last_name = 'Иванов'")).isZero();
}

@Test
@DisplayName("п.3 Другой клиент с идентификационным номером студента – сообщение об ошибке")
void anotherClientWithSameIdentificationNumber() {
    fill(TestClients.with("identificationNumber", "3140301A001PB5"));
    save();

    assertThat(error("identificationNumber")).isEqualTo("Клиент с таким идентификационным номером уже зарегистрирован");
    assertThat(count("identification_number = '3140301A001PB5'")).isEqualTo(1);
    assertThat(count("last_name = 'Иванов'")).isZero();
}

@Test
@DisplayName("п.4 Фамилия «1234» – сообщение об ошибке, в базе такой фамилии нет")
void digitsInsteadOfLastName() {
    fill(TestClients.with("lastName", "1234"));
    save();

    assertThat(error("lastName")).isEqualTo("Допустимы только буквы");
    assertThat(count("last_name = '1234'")).isZero();
}

@Test
@DisplayName("п.5 Пробел вместо имени – сообщение об ошибке, клиент не записан")
void spaceInsteadOfFirstName() {
    fill(TestClients.with("firstName", " "));
    save();

    assertThat(error("firstName")).isEqualTo("Обязательное поле");
    assertThat(count("last_name = 'Иванов'")).isZero();
}

@ParameterizedTest(name = "дата рождения «{0}»")
@CsvSource(delimiter = '|', value = {
    "не дата|Обязательное поле", // маска не пропускает буквы – поле остаётся пустым
    "31.02.2015|Такой даты не существует, формат ДД.ММ.ГГГГ",
    "29.02.2015|Такой даты не существует, формат ДД.ММ.ГГГГ",
    "15.07.20|Такой даты не существует, формат ДД.ММ.ГГГГ",
})
@DisplayName("п.6 В поле даты рождения текст, не являющийся датой")
void birthDateIsNotDate(String typed, String message) {
    fill(TestClients.with("birthDate", typed));
    save();
}

```

```

        assertThat(error("birthDate")).isEqualTo(message);
        assertThat(count("last_name = 'Иванов'")).isZero();
    }

    @ParameterizedTest(name = "{0}: «{1}»")
    @CsvSource(delimiter = '|', value = {
        "passportSeries|M|Серия — две заглавные латинские буквы",
        "passportNumber|12345|Номер паспорта — семь цифр",
        "identificationNumber|3170590A077|Формат: 7 цифр, буква, 3 цифры, 2 буквы, цифра",
        "homePhone|20145|Формат: 293-88-44",
        "mobilePhone|+375 (17) 765-43-21|Формат: +375 (29) 314-15-92, код 25, 29, 33 или 44",
        "issueDate|30.02.2018|Такой даты не существует, формат ДД.ММ.ГГГГ",
    })
    @DisplayName("н.6 Поля с маской: неполное или неверное значение")
    void maskedFields(String field, String typed, String message) {
        fill(TestClients.with(field, typed));
        save();

        assertThat(error(field)).isEqualTo(message);
        assertThat(count("last_name = 'Иванов'")).isZero();
    }

    @Test
    @DisplayName("Маска ввода отбрасывает недопустимые символы и расставляет разделители")
    void masksFilterInput() {
        type("passportNumber", "12ab34567999");
        type("passportSeries", "mp1");
        type("birthDate", "17051990");
        type("mobilePhone", "297654321");

        assertThat(value("passportNumber")).isEqualTo("1234567");
        assertThat(value("passportSeries")).isEqualTo("MP");
        assertThat(value("birthDate")).isEqualTo("17.05.1990");
        assertThat(value("mobilePhone")).isEqualTo("+375 (29) 765-43-21");
    }

    @ParameterizedTest(name = "без поля {0}")
    @ValueSource(strings = {
        "lastName", "firstName", "middleName", "birthDate", "sex", "passportSeries", "passportNumber",
        "issuedBy", "issueDate", "identificationNumber", "birthPlace", "residenceCityId", "residenceAddress",
        "registrationCityId", "maritalStatusId", "citizenshipId", "disabilityId"})
    @DisplayName("н.7 Не заполнено одно обязательное поле — сообщение об ошибке, записи нет")
    void requiredFieldIsMissing(String field) {
        Map<String, Object> client = TestClients.valid();
        client.remove(field);
        fill(client);
        save();

        assertThat(error(field)).isEqualTo("Обязательное поле");
        assertThat(count("1 = 1")).isEqualTo(6);
    }

    @Test
    @DisplayName("н.8 Только обязательные поля — клиент записан и появился в списке")
    void onlyRequiredFields() {
        fill(TestClients.onlyRequired());
        save();
        waitForNotice("Клиент добавлен");

        assertThat(driver.findElement(By.id("clients")).getText()).contains("Иванов Иван Иванович");
        assertThat(count("last_name = 'Иванов' and mobile_phone is null and monthly_income is null")).isEqualTo(1);
    }

    @Test
    @DisplayName("Редактирование и удаление клиента через web-клиент")
    void editAndDelete() {
        fill(TestClients.valid());
        save();
        waitForNotice("Клиент добавлен");

        row("Иванов Иван Иванович").findElement(By.cssSelector("button.edit")).click();
        wait.until(ExpectedConditions.attributeToBe(By.id("f-lastName"), "value", "Иванов"));
        assertThat(value("monthlyIncome")).isEqualTo("2500.50");
        type("lastName", "Сидоров");
        save();
        waitForNotice("Изменения сохранены");
        assertThat(count("last_name = 'Сидоров' and passport_number = '7654321'")).isEqualTo(1);

        row("Сидоров Иван Иванович").findElement(By.cssSelector("button.delete")).click();
        wait.until(ExpectedConditions.alertIsPresent()).accept();
        waitForNotice("Клиент удалён");
    }

```

```

        assertThat(count("last_name = 'Сидоров'")).isZero();
    }

    // ----- работа со страницей -----

    private void fill(Map<String, Object> client) {
        client.forEach((field, value) -> {
            switch (field) {
                case "sex" -> driver.findElement(By.id("f-sex-" + value)).click();
                case "pensioner" -> {
                    if (Boolean.TRUE.equals(value)) {
                        driver.findElement(By.id("f-pensioner")).click();
                    }
                }
                case "residenceCityId", "registrationCityId", "maritalStatusId", "citizenshipId", "disabilityId" ->
                    new Select(driver.findElement(By.id("f-" + field))).selectByIndex((Integer) value);
                default -> type(field, String.valueOf(value));
            }
        });
    }

    private void type(String field, String text) {
        WebElement input = driver.findElement(By.id("f-" + field));
        input.clear();
        input.sendKeys(text);
    }

    private String value(String field) {
        return driver.findElement(By.id("f-" + field)).getDomProperty("value");
    }

    private void save() {
        driver.findElement(By.id("btn-save")).click();
    }

    private String error(String field) {
        return wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("e-" + field))).getText();
    }

    private String formAlert() {
        return wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("form-alert"))).getText();
    }

    private void waitForNotice(String text) {
        wait.until(ExpectedConditions.textToBe(By.id("notice"), text));
    }

    private WebElement row(String fullName) {
        return driver.findElement(By.xpath("//table[@id='clients']/tr[td[normalize-space()=' " + fullName + "']]"));
    }

    /** Проверка «прямо в базе»: SQL-запрос к таблице client. */
    private int count(String where) {
        return jdbc.queryForObject("select count(*) from client where " + where, Integer.class);
    }
}

```

Листинг А.42 – client-service/src/test/java/by/bsuir/bank/client/DateTextValidatorTest.java

```

package by.bsuir.bank.client;

import by.bsuir.bank.client.validation.DateTextValidator;
import by.bsuir.bank.client.validation.Dates;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

import java.time.LocalDate;

import static org.assertj.core.api.Assertions.assertThat;

/** Модульные тесты проверки дат: п. 6 программы защиты. */
class DateTextValidatorTest {

    private final DateTextValidator validator = new DateTextValidator();

    @ParameterizedTest
    @ValueSource(strings = {"14.03.2001", "29.02.2016", "01.01.1900", "31.12.1999"})
    void acceptsExistingDates(String text) {

```

```

        assertThat(validator.isValid(text, null)).isTrue();
    }

    @ParameterizedTest
    @ValueSource(strings = {
        "не дата", "31.02.2015", "29.02.2015", "32.01.2000", "00.10.2000", "15.13.2000",
        "2015-02-28", "1.2.2015", "28.02.15", "31.12.1899"})
    void rejectsTextThatIsNotDate(String text) {
        assertThat(validator.isValid(text, null)).isFalse();
    }

    @Test
    void rejectsFutureDate() {
        String tomorrow = Dates.format(LocalDate.now().plusDays(1));
        assertThat(validator.isValid(tomorrow, null)).isFalse();
        assertThat(validator.isValid(Dates.format(LocalDate.now()), null)).isTrue();
    }

    @Test
    void emptyValueIsLeftToNotBlank() {
        assertThat(validator.isValid(null, null)).isTrue();
        assertThat(validator.isValid(" ", null)).isTrue();
    }
}

```

Листинг А.43 – client-service/src/test/java/by/bsuir/bank/client/TestClients.java

```

package by.bsuir.bank.client;

import by.bsuir.bank.client.dto.ClientRequest;
import com.fasterxml.jackson.databind.ObjectMapper;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/** Тестовые данные: корректно заполненная форма клиента, которую тесты портят по одному полю. */
public final class TestClients {

    public static final List<String> REQUIRED_FIELDS = List.of(
        "lastName", "firstName", "middleName", "birthDate", "sex", "passportSeries", "passportNumber",
        "issuedBy", "issueDate", "identificationNumber", "birthPlace", "residenceCityId", "residenceAddress",
        "registrationCityId", "maritalStatusId", "citizenshipId", "disabilityId", "pensioner");

    public static final List<String> OPTIONAL_FIELDS = List.of("homePhone", "mobilePhone", "email", "monthlyIncome");

    private static final ObjectMapper MAPPER = new ObjectMapper();

    /** Новый клиент, которого нет в начальном заполнении базы. */
    public static Map<String, Object> valid() {
        Map<String, Object> client = new LinkedHashMap<>();
        client.put("lastName", "Иванов");
        client.put("firstName", "Иван");
        client.put("middleName", "Иванович");
        client.put("birthDate", "17.05.1990");
        client.put("sex", "М");
        client.put("passportSeries", "МР");
        client.put("passportNumber", "7654321");
        client.put("issuedBy", "Фрунзенское РУВД г. Минска");
        client.put("issueDate", "20.06.2018");
        client.put("identificationNumber", "3170590A077PB4");
        client.put("birthPlace", "г. Минск");
        client.put("residenceCityId", 1);
        client.put("residenceAddress", "ул. Притыцкого, д. 29, кв. 3");
        client.put("homePhone", "201-45-67");
        client.put("mobilePhone", "+375 (29) 765-43-21");
        client.put("email", "ivanov@example.by");
        client.put("registrationCityId", 1);
        client.put("maritalStatusId", 1);
        client.put("citizenshipId", 1);
        client.put("disabilityId", 1);
        client.put("pensioner", false);
        client.put("monthlyIncome", 2500.50);
        return client;
    }

    public static Map<String, Object> with(String field, Object value) {
        Map<String, Object> client = valid();
        client.put(field, value);
        return client;
    }
}

```

```
}

public static Map<String, Object> onlyRequired() {
    Map<String, Object> client = valid();
    OPTIONAL_FIELDS.forEach(client::remove);
    return client;
}

public static ClientRequest request(Map<String, Object> fields) {
    return MAPPER.convertValue(fields, ClientRequest.class);
}

public static String json(Map<String, Object> fields) throws Exception {
    return MAPPER.writeValueAsString(fields);
}

private TestClients() {
}
}
```